



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ
ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΕΡΓΑΣΤΗΡΙΟ ΒΑΣΕΩΝ ΔΕΔΟΜΕΝΩΝ

Ακαδημαϊκό Έτος 2022-2023

ΤΕΧΝΙΚΗ ΑΝΑΦΟΡΑ

PROJECT

ΚΙΟΥΡΤ-ΜΕΧΜΕΤΑΛΗ ΑΧΜΕΤ

A.M.: 1084672

Έτος Φοίτησης:3ο

Πάτρα 2022-23

1^ο Κεφάλαιο

Νέοι Πίνακες

Πίνακας IT: Είναι ένας Πίνακας που και αυτός είναι worker και συνδέεται με το πίνακα worker.

IT_AT: Είναι ο αριθμός ταυτότητας του **Υπευθύνου Πληροφορικής** και συνδέεται με το **wrk_AT** του πίνακα worker. Χρησιμοποιούμε το **CHAR(10)** που το έχει ίδιο και το **wrk_at** του worker.

IT_password: Έχει χαρακτήρα **CHAR(10)** που είναι **NOT NULL**, δηλαδή δεν μπορεί να είναι άδειο. Επίσης περιλαμβάνει **DEFAULT** το **"password"** ως κώδικα σύμφωνα και με την εκφώνηση.

start_date: Είναι μία εντολή που έχει χαρακτηριστικό **DATETIME**, δηλαδή ημερομηνία και είναι **NOT NULL** αφού θα υπάρχει για κάθε εργαζόμενο μία μέρας αρχής για την εργασία που ασχολείται.

end_date: Διαφορετικά από το **start_date** δεν έχουμε **NOT NULL** αλλά έχουμε το χαρακτηριστικό **DATETIME** όπως και στο **start_date**.

Στην συνέχεια δηλώνουμε το **IT_AT** ως **PRIMARY KEY** που και αυτό συνδέεται με το **worker** ως foreign key όπως αναφερθήκαμε και παραπάνω.

Πίνακας offers: Περιέχει τις προσφορές σε ταξίδια αλλά και με προορισμούς.

offer_tr_id: Βάζουμε ένα ID στην προσφορά που έχουμε δημιουργήσει για να είναι διακριτά τα δεδομένα αλλά σύμφωνα και με την εκφώνηση. Σε μορφή **INT(11)**.

offer_st_date: Η ημερομηνία της αρχής της περιόδου που μπορεί να πραγματοποιηθεί κάποιο ταξίδι. Έχει την εντολή **DATETIME** και δεν είναι η διάρκεια κάποιου ταξιδιού αλλά η αρχική ημερομηνία που θα ισχύει η προσφορά.

offer_end_date: Έχει την εντολή **DATETIME** και περιγράφει μέχρι πότε μπορεί να ισχύει κάποια προσφορά.

cost: Περιλαμβάνει το κόστος της προσφοράς σε μορφή **FLOAT(7,2)**

offer_dst_id: Είναι **INT(11)** όπως και το **dst_id** του destination που είναι **FOREIGN KEY** στο destination key και δίνει δεδομένα στο **dst_id** για να ξέρουμε ποιους προορισμούς περιλαμβάνει η προσφορά.

Ως **PRIMARY KEY** έχουμε το **offer_tr_id** για να τον έχουμε μοναδικό σύμφωνα με την εκφώνηση, βέβαια θα μπορούσαμε να προσθέσουμε και **unique** στο συγκεκριμένο. Επίσης βάζουμε και το **offer_dst_id** ως **PRIMARY KEY** να μπορούμε να έχουμε πολλά destinations σε ένα **trip** ακόμα και να είναι **offer** το **trip**.

Που στο τέλος του πίνακα έχουμε το **offer_dst_id** ως **FOREIGN KEY** και reference στο **dst_id** του **destination** όπως έχουμε αναφερθεί.

Πίνακας reservation_offers: Ένας πίνακας που περιλαμβάνει τα στοιχεία των πελάτων για το πίνακα **offers**.

res_off_name : Είναι σε μορφή **VARCHAR(20)**, περιλαμβάνει τα ονόματα των πελάτων που έχουν κάνει κράτηση.

res_off_name : Είναι σε μορφή **VARCHAR(20)**, περιλαμβάνει τα επίθετα των πελάτων που έχουν κάνει κράτηση.

res_off_id : Σε μορφή **INT(12)** που είναι **NOT NULL** και **AUTO INCREMENT** μετράει τους πελάτες με την σειρά που τα έχουμε δηλώσει και βάζει ένα αριθμό ως ID. Έχουμε ορίσει το ID σαν **AUTO_INCREMENT = 250000** για να έχουμε μία καλύτερη εικόνα.

res_offer_tr_id: Σε μορφή **INT(11)** που θα συνδέεται και με το **offer_tr_id** του **offers** για να καταχωρήσουμε ποια προσφορά έχει επιλέξει ο συγκεκριμένος πελάτης.

deposit_cost: Το έχουμε ορίσει σε **FLOAT(7,2)** που απλά βάζουμε κάποιο κόστος σε **trip** ανάλογα με την εκφώνηση.

Στην συνέχεια, έχουμε ορίσει το **PRIMARY KEY** σε **res_off_id** για να είναι μοναδικός και ως **FOREIGN KEY** το **res_offer_tr_id** που είναι reference στο **offer_tr_id** του **offers**.

Σχετικά με τα δεδομένα του **reservation_offers!!**

Επειδή τα DATA των **reservation_offers** είναι πάρα πολλά και θα ήταν απίθανο να το βάλουμε στην αναφορά, έχουμε βάλει τα DATA σε ένα txt με όνομα **reservation_offers.txt**

Πίνακας logg:

log_AT: Είναι σε μορφή Auto Increment, το χρησιμοποιούμε για να δούμε ποιος έχει κάνει την υλοποίηση πρώτα.

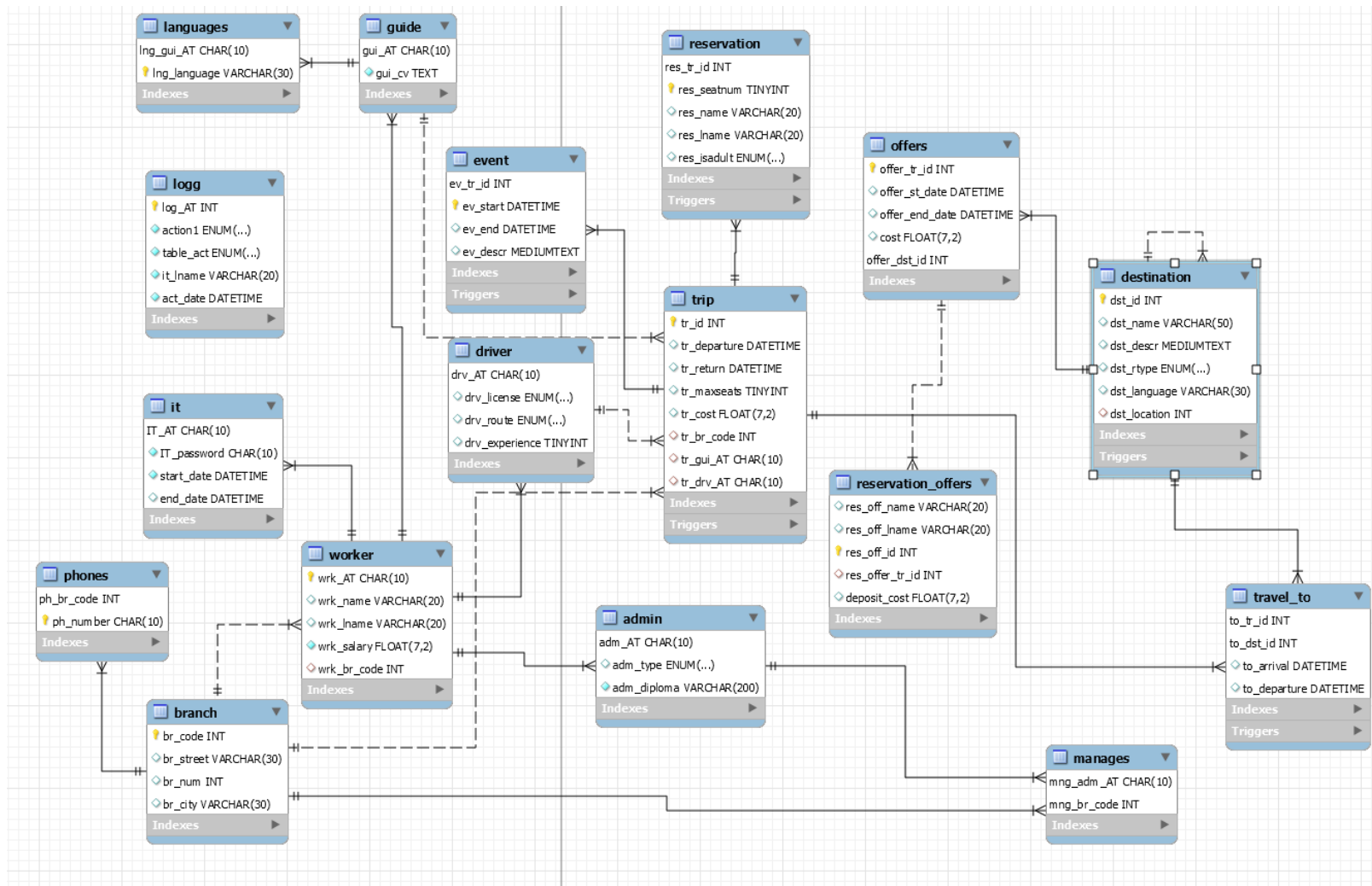
action1: Σε μορφή **ENUM** που γράφουμε την ενέργεια που υλοποιούμε για να το γράψουμε στην συνέχεια στο πίνακα.

table_act: Ξανά **ENUM** με ίδια λογική του επάνω με διαφορά διαλέγουμε το πίνακα.

it_lname: Το επώνυμο του IT που έχει την τιμή **VARCHAR(20)** όπως ο worker για σύνδεση.

act_date: Είναι **DATETIME** που βγάζει την ημερομηνία, για να συνδεθεί με την συνάρτηση **NOW()** που θα έχουμε την στιγμή που γίνεται η υλοποίηση.

Σχεσιακό Διάγραμμα travel_agency



Αρχικός Κώδικας

Ο κώδικας που έχουμε υλοποιήσει μέσα στο εξάμηνο και η βάση όλου του κώδικα.

```
DROP DATABASE if exists travel_agency;
CREATE DATABASE travel_agency;
USE travel_agency;

create table branch(
    br_code int(11) not null unique auto_increment,
    br_street varchar(30),
    br_num int(4),
    br_city varchar(30),
    primary key(br_code)
)engine=InnoDB;

create table phones(
    ph_br_code int(11) not null,
    ph_number char(10),
    primary key(ph_br_code,ph_number),
    foreign key (ph_br_code) references branch(br_code)
    on delete cascade on update cascade
)engine=InnoDB;

/*Θα μπορούσαμε να βάλουμε στο wrk_AT auto_increment αλλά είναι
char*/
create table worker(
    wrk_AT char(10) not null,
    wrk_name varchar(20),
    wrk_lname varchar(20),
    wrk_salary float(7,2) not null,
    wrk_br_code int(11),
    primary key(wrk_AT),
    foreign key (wrk_br_code) references branch(br_code)
    ON DELETE CASCADE ON UPDATE CASCADE
)engine=InnoDB;

create table admin(
    adm_AT char(10) not null,
    adm_type enum('LOGISTICS','ADMINISTRATIVE','ACCOUNTING'),
    adm_diploma varchar(200) not null,
    primary key(adm_AT),
    foreign key(adm_AT) references worker(wrk_AT)
    ON DELETE CASCADE ON UPDATE CASCADE
)engine=InnoDB;

create table manages(
    mng_adm_AT char(10) not null,
    mng_br_code int(11) not null,
    primary key(mng_adm_AT, mng_br_code),
    foreign key(mng_adm_AT) references admin(adm_AT)
    ON DELETE CASCADE ON UPDATE CASCADE,
    foreign key (mng_br_code) references branch(br_code)
    ON DELETE CASCADE ON UPDATE CASCADE
)engine=InnoDB;

create table guide(
    gui_AT char(10) not null,
    gui_cv text not null,
    primary key (gui_AT),
```

```

foreign key (gui_AT) references worker(wrk_AT)
on delete cascade on update cascade
)engine=InnoDB;

create table languages(
    lng_gui_AT char(10) not null,
    lng_language varchar(30) not null,
    primary key (lng_gui_AT,lng_language),
    foreign key (lng_gui_AT) references guide(gui_AT)
    on delete cascade on update cascade
)engine=InnoDB;

create table driver(
    drv_AT char(10) not null,
    drv_license enum('A','B','C','D'),
    drv_route enum ('LOCAL','ABROAD'),
    drv_experience tinyint(4),
    primary key (drv_AT),
    foreign key (drv_AT) references worker(wrk_AT)
    on delete cascade on update cascade
)engine=InnoDB;

create table trip(
    tr_id int(11) not null unique,
    tr_departure datetime unique,
    tr_return datetime unique,
    tr_maxseats tinyint(4),
    tr_cost float(7,2),
    tr_br_code int(11),
    tr_gui_AT char(10),
    tr_drv_AT char(10),
    primary key (tr_id),
    foreign key (tr_drv_AT) references driver(drv_AT)
    on delete cascade on update cascade,
    foreign key (tr_br_code) references branch(br_code)
    on delete cascade on update cascade,
    foreign key (tr_gui_AT) references guide(gui_AT)
    on delete cascade on update cascade
)engine=InnoDB;

create table destination(
    dst_id int(11) not null,
    dst_name varchar(50),
    dst_descr text(65535),
    dst_rtype enum('LOCAL','ABROAD'),
    dst_language varchar(30),
    dst_location int(11) ,
    primary key (dst_id),
    foreign key (dst_location) references destination(dst_id)
    on delete cascade on update cascade
) engine=InnoDB;

create table travel_to(
    to_tr_id int(11) not null,
    to_dst_id int(11) not null,
    to_arrival datetime,
    to_departure datetime,
    primary key(to_tr_id,to_dst_id),
    foreign key (to_tr_id) references trip(tr_id)
    on delete cascade on update cascade,
    foreign key (to_dst_id) references destination(dst_id)

```

```

on delete cascade on update cascade
)engine=InnoDB;

create table reservation(
  res_tr_id int(11) not null,
  res_seatnum tinyint(4) not null,
  res_name varchar(20),
  res_lname varchar(20),
  res_isadult enum('ADULT','MINOR'),
  primary key (res_tr_id,res_seatnum),
  foreign key (res_tr_id) references trip(tr_id)
on delete cascade on update cascade
)engine=InnoDB;

create table event(
  ev_tr_id int(11) not null ,
  ev_start datetime not null,
  ev_end datetime,
  ev_descr text(65535) ,
  primary key(ev_tr_id,ev_start),
  foreign key (ev_tr_id) references trip(tr_id)
on delete cascade on update cascade
)engine=InnoDB;

/* Ξεκινάμε από ένα specific number*/
ALTER TABLE branch AUTO_INCREMENT = 20000;

INSERT into branch VALUES
(NULL,'Oak Avenue','12','Temple City'),
(NULL,'Liberty Lane','107','Washington'),
(NULL,'Fortune Passage','18','Phoenix'),
(NULL,'Corporation Avenue','0013','New York'),
(NULL,'Long Street','083','New York');

/* Έχουμε auto_increment άρα πρέπει να μην ξεκινήσουμε από τυχαία
τιμή*/
INSERT into phones VALUES
('20000','9518215849'),
('20000','9513593396'),
('20002','9512213269'),
('20001','2566745200'),
('20001','6788209200'),
('20004','6307865111'),
('20004','5024214111'),
('20003','5057028444'),
('20003','4096791444'),
('20002','3096299666');

INSERT into worker VALUES
('98872BAA72','Sandra','Koch','70500.50','20000'), /*admin
20000*/
('8147862271','Declan','Hogan','20000.50','20000'),/*guide*/
('B1855A1B76','John','Stewart','50125.00','20002'), /*driver*/
('710CC7A849','Thomas','Butler','95500.00','20001'), /*driver*/
('6C738794A1','Lucia','Bray','27000.50','20001'), /*admin
20001*/
('6C5C004CC3','Marco','Hart','60000.40','20004'), /*guide*/

```

```

        ('88244C2507','Gerard','Mccoy','55500.50','20004'), /*admin
20004*/
        ('197B244051','Arran','Bender',95500.20,'20003'), /*admin 20003*/
        ('251A2C4988','Haaris','Lucas',25000.00,'20003'), /*driver*/
        ('745066107B','Lilly-Rose','Wilkerson','98000.50','20002'), /*New
added under this column*/ /*admin 20002*/
        ('737787B8B3','Samson','Hampton','78500.50','20000'), /*admin
20000*/
        ('AB74CA2361','Jerry','Wallace','43000.50','20000'), /*driver*/
        ('CA3108032C','Anoushka','Bridges','63125.00','20002'), /*guide*/
        ('820C85C87C','Kate','Keith','23500.00','20001'), /*admin 20001*/
        ('831482CC74','Jimmy','Humphrey','76000.50','20001'), /*guide*/
        ('2161C52254','Rose','Cruz','88000.40','20004'), /*admin 20004*/
        ('72A3AB5128','Earl','Kemp','90500.50','20004'), /*driver*/
        ('757006C2BB','Ellie-Mae','Barr','30500.20','20003'), /*admin
20003*/
        ('8B207900AB','Macie','Cobb','34000.00','20003'), /*guide*/
        ('5258C7B8C6','Elodie','Carlson','87000.50','20002'), /*admin
20002*/
        /*guide*/
        ('8C8166410C','Dennis','Branch','40000.40','20000'),
        ('0BB4BC6AB2','Claire','Coleman','60000.50','20001'),
        ('43A216CBB4','Heidi','Scott','55000.00','20002'),
        ('15C3A8C1C0','Andreas','Potts','47500.50','20003'),
        ('C59C685619','Gunilla','Bowen','45000.50','20004'),
        /*driver*/
        ('97C9685C31','Kaylum','Gallagher','35500.50','20000'),
        ('68B9B9909B','Jackson','Davila','43000.60','20001'),
        ('369935A4C4','Corey','Orr','34000.50','20002'),
        ('0337628780','Nataniel','Shepherd','35000.50','20003'),

        /*admin*/
        ('621B027416','Caoimhe','House','25000.00','20000'),
        ('26873C4924','Stephanie','Park','30500.50','20001'),
        ('4399691360','Scarlet','Chaney',45000.50,'20002'),
        ('4C045A649A','Olivia','Camacho',38000.60,'20003'),
        ('93C4BBCB6C','Elise','Meyer','40000.40','20004'),

        /* New workers που θα είναι driver για το πρώτο procedure δεν θα
βάλουμε για το ένα branch νέο driver για να δούμε αν θα δουλεύει το
procedure*/
        ('03EC1106E8','Layton','Gibbons','25000.00','20000'),
        ('643CB89970','Josh','Molina','30000.50','20001'),
        ('D0D31BBC92','Alexander','Knapp','32000.00','20002'),
        ('4BD720A79B','Gilbert','Campbell','27000.50','20003'),

        /*IT*/
        /*NULL*/
        ('12345ABCDE','Jimmy','Butler','27000.50',20000),
        ('12345ABCDF','John','Lennon','19000.45',20001),
        ('12345ABCDG','Margot','Andrew','34000.50',20002),
        ('12345ABCDH','Jeff','Clark','45600.54',20003),
        ('12345ABCDI','Ryan','Phillips','45450.70','20004'),

        /* Δύο άτομα που δουλεύουν την ίδια χρονική στιγμή*/
        ('12345ABSDF','Laurie','Adkins','23000.56',20000),
        ('12345ABPLO','Tom','Hall','30000.50',20001),

        /*not NULL*/
        ('12345ABCKL','Bryan','Frost','70000.50',20000),
        ('12345ABCMN','Olive','Larson','80000.50',20001),

```



```

('12345ABCOP','Tamara','Wood','60000.60',20002),
('12345ABCRS','Verity','Evans','76000.50',20003),
('12345ABCTY','John','Lam','80000.60',20004);

```

INSERT into admin VALUES

```

('98872BAA72','LOGISTICS','Bachelor in Logistics Engineering
Fontys University of Applied Sciences Venlo, Netherlands'), /*20000*/
('737787B8B3','ADMINISTRATIVE','B.S.B.A. Management Nova
Southeastern University Undergraduate Programs Fort Lauderdale,
USA'),
('820C85C87C','ADMINISTRATIVE','B.A. Public Administration
University of Tennessee Knoxville'), /*20001*/
('6C738794A1','ACCOUNTING','Bachelor of Science - Accounting
Eastern University'),
('5258C7B8C6','ADMINISTRATIVE','B.A./B.S. in Transportation and
Logistics Management University of Wisconsin Superior Superior,
USA'), /*20002*/
('745066107B','ACCOUNTING','Bachelor of Business - Major in
Accounting Melbourne Institute Of Technology Melbourne, Australia'),
('197B244051','ADMINISTRATIVE','Bachelor of Business
Administration Degree Capilano University North Vancouver, Canada'),
/*20003*/
('757006C2BB','LOGISTICS','Bachelor of Arts (Honours) Business
Logistics and Transport Management (Top-up) LSBF Singapore
Singapore'),
('88244C2507','ADMINISTRATIVE','BSC in Business Administration
Worcester State University Worcester, USA'), /*20004*/
('2161C52254','ACCOUNTING','Management: Accounting Cazenovia
College Cazenovia, USA'),
/*new admins*/
('621B027416','ACCOUNTING','BA (Hons) Business Management
(Accounting and Finance) with Foundation Year, Arden University'),
('26873C4924','LOGISTICS','Bachelor in International Logistics
Management, NHL Stenden University of Applied Sciences'),
('4399691360','LOGISTICS','Bachelor of Commerce in Operations
Management, University of Alberta'),
('4C045A649A','ACCOUNTING','Bachelor in Accounting, Carroll
University'),
('93C4BBCB6C','LOGISTICS','BSc (Hons) in Finance, Operations
Research, Management and Statistics, University of York');

```

INSERT into manages VALUES

```

('737787B8B3','20000'),
('820C85C87C','20001'),
('5258C7B8C6','20002'),
('88244C2507','20004'),
('197B244051','20003');

```

INSERT into guide VALUES

```

/*Serbian English*/
('8147862271','Declan is a licensed tourist guide since 2011, and
has a background in Balkan regional studies, focusing mostly on
history,
political philosophy and international relations. Being of mixed
English and Serbian heritage,
he has often had to operate as a sort of conduit between cultures,
so as to avoid misunderstandings and

```

family members getting "lost in the translation". When he was given to the opportunity to become a professional guide, it felt rather familiar to him and he leapt at the chance. Declan loves receiving interesting questions by guests from around the world, but hates whitewashing the answers. '), /*20000*/

('831482CC74','Jimmy Humphrey is a half Australian, half Icelandic musician/guide who has been guiding since the millennium. He specialises in trekking and backpacking tours and he loves to find new routes and trying new trails. His favourite place in the world is Grænafjall, a beautiful and isolated mountain with lots of wild vegetation. Even blueberries can be found there spite of being at high altitude and surrounded by glaciers. You might find Tomas on any of our Greenland tours or backpacking between Núpsstaðarskógar and Skaftafell. He has experience of guiding a lot of different locations in the world. '), /*20001*/

('CA3108032C',' Anoushka was born and raised in downtown Hamburg but during the summer months she would travel abroad or stay at her farm close to the volcano Mt. Hekla.

She studied Tourist Management and has lived in Norway, Germany, Sweden and Madrid. She started as a photography tour guide because it allowed her to combine her 20 years

of freelance photography with her need to be out in nature. Guiding has also been a great opportunity for her to use the languages that she has learned

and be out in a beautiful surrounding with interesting people. She has been a part of the Swiss Mountain Guides team since 2011 and she has been guiding for over 15 years.

She is a certified Wilderness First Responder (WFR).

Alongside guiding, she teaches photography at an art academy and still works freelance, both in a studio and doing landscape photography.

She feels lucky to have done a lot of traveling but she still has a lot of places on her bucket list.

She is mostly drawn to photogenic places or areas that allow her to hike, ski, run or cycle.

She would love to hike in Nepal, run the Mount Blanc, cycle the Ruta Austral in Chile or Peru, as well as photograph South Africa, Japan, India, Mexico and Argentina. '), /*20002*/

('8B207900AB','Macie comes from the Netherlands, from a small village deep in the woods. He became a guide in order to work in nature and with people and previous to joining IMG in spring 2017

he worked as a ski instructor in the biggest ski resort in Czech Republic and also in Austria,

and as a ski instructor in Japan. In his free time he loves skiing, of course, running and cycling and as a junior

he was the double handed champion of Netherlands in chess!

His favourite place is in Peru the beautiful and peaceful Machu Picchu and he has Svalbarð in Iceland on his bucket list. '), /*20003*/

('6C5C004CC3','Marco was born close the mountains of Italy and home is Bologna. He has a background in industrial rigging work, hanging off sky scrapers or fixing wind turbines.

Marco also enjoys ski touring. He is relatively new to guiding and 2017 is his first season with IMG,

but he has spent 20 years climbing extensively in Europe, South America and in the Himalaya.']), /*20004*/

('8C8166410C','Dennis has always loved traveling, an off-shoot of being born on the Isle of Mann into a British-Italian family. Dennis grew up in Italy, but spent his summers on that island in the Irish sea, with the result that he is perfectly bilingual and bi-cultural. Indeed, he has a passion for intercultural experiences and completed postgraduate studies in international cooperation and development. He has left his footprints in the deserts of Morocco and the jungles of Peru, climbed the Pyrenees in Spain and volcanoes in Mexico. He now enjoys walking in the forests of Tuscany, where he lives. His passion for nature and sustainability recently led him to take part in a business venture related to organic farming, which is now his main activity when he's not working in tourism or teaching languages. Dennis guides our tours in the Dolomites (Alta Via and Dolomites Grand Traverse), Tuscany, the Cinque Terre, Andalusia, Amalfi and Piedmont and he guided the tours that we provided to REI Adventures in the Dolomites, Cinque Terre and Tuscany. Dennis is fluent in English, Italian, Spanish and Manx.']), /*20000*/

('0BB4BC6AB2','Claire has a PhD in French and Comparative Literature but happily abandoned academia years ago to become a guide. Her zest for life and wanderlust have taken her all over the world, from Albania to Zanzibar, and she has never lost her passion for the job. Claire won the REI TOP GUIDE award in 2005 and is fluent in French and Italian. She lives in Florence when she is not guiding our tours in Italy, France, Spain, Croatia, Greece and Slovenia.']), /*20001*/

('43A216CBB4','Heidi was born in Gibraltar but was raised in Plymouth, in the U.K. She studied Social Policy and Administration at Brighton University and worked in Social Care for a few years, but eventually embarked on a round-the-world trip during which she spent 7 months in New Zealand. She fell in love with the country and returned to live there in 2013, making a new life in Christchurch where she is busy raising two daughters. She now works part-time for us taking payments and issuing trip agreements. '), /*20002*/

('15C3A8C1C0','Andreas has studied biology at the Swedish University of Agricultural Science. In 2003 he started working as a guide and has been guiding ever since. He has also been working on different research projects and research stations, from gerbills in the Kalahari desert to green sea turtles in Costa Rica. '), /*20003*/

('C59C685619','Gunilla started her career as a guide as her love for the white, remote and pristine places of the polar regions became too strong to materialise into anything but a lifelong passionate career as a polar guide. When not guiding expeditions, she is fond of skiing and photographing wildlife. '); /*20004*/

INSERT into languages **VALUES** /* 24 εγγραφές*/
('8147862271', 'English'),

```

('8147862271','Serbian'),
('831482CC74','English'),
('831482CC74','Icelandic'),
('831482CC74','Italian'),
('CA3108032C','English'),
('CA3108032C','German'),
('8B207900AB','English'),
('8B207900AB','Dutch'),
('6C5C004CC3','Italian'),
('6C5C004CC3','English'),
/*new*/
('8C8166410C','Italian'),
('8C8166410C','Spanish'),
('8C8166410C','English'),
('0BB4BC6AB2','French'),
('0BB4BC6AB2','Italian'),
('0BB4BC6AB2','English'),
('43A216CBB4','English'),
('43A216CBB4','Vietnamese'),
('15C3A8C1C0','Swedish'),
('15C3A8C1C0','English'),
('C59C685619','Portuguese'),
('C59C685619','Indian'),
('C59C685619','English');

INSERT into driver VALUES
('B1855A1B76','B','LOCAL','95'), /*1202*/
('710CC7A849','D','ABROAD','78'), /*1201*/
('251A2C4988','A','LOCAL','35'), /*1203*/
('AB74CA2361','C','ABROAD','18'), /*1300*/
('72A3AB5128','A','ABROAD','37'), /*1204*/

('97C9685C31','D','LOCAL','12'), /*1200*/
('68B9B9909B','B','LOCAL','43'), /*1301*/
('369935A4C4','C','ABROAD','32'), /*1302*/
('0337628780','A','LOCAL','43'), /*1303*/

/*New drivers που έχουμε δημιουργήσει στο worker για το πρώτο
procedure*/
('03EC1106E8','A','ABROAD','11'),
('643CB89970','B','ABROAD','19'),
('D0D31BBC92','C','ABROAD','24'),
('4BD720A79B','D','ABROAD','26');

INSERT into trip VALUES /* Έχουμε σβήσει ένα data για το Πρώτο
Procedure που θα το προσθέτει */
('1200','2019-12-22 21:30:30','2020-01-03
18:00:00','110','349.90','20000','8147862271','97C9685C31'), /*20000
Κάθε εργαζόμενος δουλεύει μόνο ένα υποκατάστημα άρα δεν θα μπορούμε
να έχουμε διαφορετικό guide ή driver από άλλο υποκατάστημα*/
('1300','2018-07-30 06:00:30','2018-08-14
05:00:00','89','224.50','20000','8C8166410C','AB74CA2361'),

('1201','2023-01-04 09:00:00','2023-02-12
21:15:00','50','320.99','20001','831482CC74','710CC7A849'), /*20001*/
('1301','2023-02-13 22:00:00','2023-03-04
15:30:00','105','1240.50','20001','0BB4BC6AB2','68B9B9909B'),

('1202','2021-02-18 23:00:00','2021-03-02
08:00:00','98','50.50','20002','CA3108032C','B1855A1B76'), /*20002*/

```

```

('1302','2017-06-19 07:00:00','2017-07-02
06:00:00','75','1119.99','20002','43A216CBB4','369935A4C4'),

('1203','2024-05-12 18:00:00','2024-06-02
23:00:00','124','34.00','20003','8B207900AB','251A2C4988'), /*20003*/
('1303','2016-12-22 09:00:00','2017-01-15
12:00:00','60','125.80','20003','15C3A8C1C0','0337628780'),

('1204','2022-08-25 07:00:00','2022-09-18
15:00:00','90','560.50','20004','6C5C004CC3','72A3AB5128'); /*20004*/

/* Εκκινάμε από ένα specific number
ALTER TABLE destination AUTO_INCREMENT = 100;*/

```

INSERT into destination VALUES

```

('99','USA','The United States of America is the third largest
country in size and nearly the third largest in terms of population.
Located in North America, the country is bordered on the west by
the Pacific Ocean and to the east by the Atlantic Ocean.
Along the northern border is Canada and the southern border is
Mexico.','LOCAL','English',NULL),/*COUNTRY*/
('100','Detroit ','DETROIT, the largest city in the state of
Michigan, lies on the northwest bank of the Detroit River and on Lake
St. Clair,
between Lakes Huron and Erie. Downtown Detroit sits at the lake
edge and is packed with things to do, as well as restaurants, shops,
and interesting neighborhoods like Greektown.',
'LOCAL','English','99'),
('101','Alpena ','ALPENA is home to one of the largest shale
quarries, Lafarge North America - Alpena Plant. Shale and limestone
are extracted from the earth and mixed with other ingredients
to make raw cement which is shipped out from Alpena via freighters
on the Great Lakes.',
'LOCAL','English','99'),
('102','Arcadia ','ARCADIA is a water heaven delight offering sandy
beaches, Lake Michigan access channel,
excellent fishing and boating. Home to Arcadia Bluffs Golf Course
with Lake Michigan views. Rock collectors will find beaches a great
place to hunt for Petoskey Stones.',
'LOCAL','English','99'), /*1 Νομίζω δεν χρειάζεται!*/

('103','SPAIN','Spain occupies most of the Iberian Peninsula,
stretching south from the Pyrenees Mountains to the Strait of
Gibraltar, which separates Spain from Africa.
To the east lies the Mediterranean Sea, including Balearic Islands.
Spain also rules two cities in North Africa and the Canary Islands
in the Atlantic.','ABROAD','Spanish',NULL), /*COUNTRY*/
('104','Barcelona','So, What is Barcelona famous for? Barcelona is
a Mediterranean city with excellent beaches and good weather.
Art, gastronomy, and sports excel exceptionally. Gaudi
architecture, 30 Michelin stars restaurants (2023), and the Barcelona
Football Club (soccer) have made
the city a world-famous travel
destination.','ABROAD','Spanish','103'),
('111','Madrid','The Spanish capital is one of the most populous in
the European Union, a cosmopolitan city where people of over 180
nationalities live together.

```

Expatriates feel at home as they easily make friends with Madrid
has
amiable residents and adapt to the local culture, enjoying this
open, inclusive city.', 'ABROAD', 'Spanish', '103'), /*2*/

('105', 'ITALY', 'Italy, country of south-central Europe, occupying a
peninsula that juts deep into the Mediterranean Sea. Italy comprises
some of the most varied and scenic landscapes on Earth
and is often described as a country shaped like a boot. At its
broad top stand the Alps, which are among the most rugged
mountains.', 'ABROAD', 'Italian', NULL), /*3*/ /*COUNTRY*/

('106', 'California', 'Vibrant cities, beaches, amusement parks, and
natural wonders like nowhere else on Earth make California an
intriguing land of possibilities for travel.
The gateway cities of San Francisco and Los Angeles are home to
some of the most well-known sites, from the Golden Gate Bridge to
Hollywood and Disneyland.',
'LOCAL', 'English', '99'), /*4*/

('107', 'New York', 'New York is composed of five boroughs -
Brooklyn, the Bronx, Manhattan, Queens and Staten Island - is home to
8.4 million people who speak more than 200 languages,
hail from every corner of the globe, and, together, are the heart
and soul of the most dynamic city in the
world.', 'LOCAL', 'English', '99'), /*5*/

('108', 'VIETNAM', 'Vietnam is a long, narrow nation shaped like the
letter s. It is in Southeast Asia on the eastern edge of the
peninsula known as Indochina.
Its neighbors include China to the north and Laos and Cambodia to
the west. The South China Sea lies to the east and
south.', 'ABROAD', 'Vietnamese', NULL), /*COUNTRY*/

('109', 'Hanoi VIETNAM', 'Hanoi, located on the banks of the Red
River, is one of the best and ancient capital. The moment you arrive
in the Vietnamese capital, you will find streets within
the city centre where well-preserved colonial buildings, ancient
pagodas, and unique museums stand
proud.', 'ABROAD', 'Vietnamese', '108'),

('110', 'Ho Chi Minh City VIETNAM', 'Ho Chi Minh City is the largest
city, business and financial hub of Vietnam. Also known as Saigon, it
has a prominent history going back hundreds of years.
There are plenty of museums showcasing the dark wartime history and
classic colonial architecture of country built by former French
rulers.', 'ABROAD', 'Vietnamese', '108'); /*6*/

```

INSERT into travel_to VALUES
/*('1200', '99', '2019-12-22 21:30:30', '2020-01-03 18:00:00'),*/
/*Τελικά δεν χρειάζεται να προσθέτουμε τα Country αν έχουμε ένα trip
που μας εξηγεί σε ποια destinations θα είναι το ταξίδι*/
('1200', '100', '2019-12-22 21:30:30', '2020-12-27 08:00:00'),
('1200', '107', '2019-12-27 15:30:30', '2020-01-03 18:00:00'),
('1200', '102', '2019-12-27 18:30:30', '2020-01-03 19:00:00'), /*15-8-
23*/
('1200', '101', '2019-12-27 17:30:30', '2020-01-03 18:00:00'), /*15-8-
23*/
('1200', '106', '2019-12-27 16:30:30', '2020-01-03 18:00:00'), /*15-8-
23*/

/* ('1300', '103', '2018-07-30 06:00:30', '2018-08-14 05:00:00'), */

```

```

('1300','104','2018-07-30 06:00:30','2018-08-07 15:00:00'),
('1300','111','2018-08-07 21:30:30','2018-08-14 05:00:00'),

('1201','105','2023-01-04 09:00:00','2023-02-12 21:15:00'), /*To
ταξίδι περιλαμβάνει μόνο το Italy άρα πρέπει να βάλουμε το Italy στο
travel_to αφού δεν έχουμε πχ. Rome,Milan,Bologna κλπ.

/*('1301','99','2023-02-13 22:00:00','2023-03-04 15:30:00'), */
('1301','107','2023-02-13 22:00:00','2023-02-27 08:30:00'),
('1301','106','2023-03-01 23:30:00','2023-03-04 15:30:00');

/*Έχουμε δύο primary key με αποτέλεσμα σε ένα trip να μην γίνεται
κράτηση σε μία συγκεκριμένη θέση αλλά να μπορεί να γίνεται μία
συγκεκριμένη θέση σε διαφορετικά ταξίδια*/
INSERT into reservation VALUES
('1200','15','Vanessa','Proctor','ADULT'),
('1200','16','Luther','Proctor','MINOR'),
('1200','01','Lucinda','Roach','ADULT'),
('1200','10','Marvin','Morgan','ADULT'),
('1200','09','Edwin','Terrell','MINOR'),

('1300','32','Zach','Duffy','ADULT'),
('1300','21','Aleksander','Morris','MINOR'),
('1300','09','Tara','Case','ADULT'),

('1201','17','Elspeth','Schneider','MINOR'),
('1201','50','Denzel','Smith','ADULT'),
('1201','39','Georgiana','Franco','ADULT'),
('1201','02','Haris','Lamb','ADULT'),

('1301','90','Robert','Gross','ADULT'),
('1301','58','Mariam','King','ADULT');

/*'2018-07-30 06:00:30','2018-08-14 05:00:00' 1300*/
/*'2019-12-22 21:30:30','2020-01-03 18:00:00' 1200*/
/*'2023-01-04 09:00:00','2023-02-12 21:15:00' 1201*/
/*'2023-02-13 22:00:00','2023-03-04 15:30:00' 1301*/
/*Τα event που έχουμε δημιουργήσει πρέπει να έχουν ημερομηνίες
ενδιάμεσα του trip */
INSERT into event VALUES

('1200','2019-12-23 12:00:00','2019-12-23 17:00:00','Detroit
Institute of Arts
Considered to house one of the best art collections in the United
States, the museum showcases everything fro
m mummies to modern art and African masks to Monets in its
outstanding collection of over 65,000 works. Don not miss the
Center for African American Art, a gallery in the DIA that showcases
400 pieces, in various media, by African American artists. '),
('1200','2019-12-28 18:00:00','2019-12-28 21:00:00','Koch Dinosaur
Wing, the Hall of Saurischian Dinosaurs displays fossils from one of
the two major
groups of dinosaurs. Saurischians are characterized by grasping
hands,
in which the thumb is offset from the other fingers. This hall
features the imposing mounts of Tyrannosaurus rex and Apatosaurus. '),

('1300','2018-07-30 12:00:30','2018-07-30 15:00:00','It is the
Picasso Museum! The permanent collection featu

```

res nearly 4,000 pieces, mainly showing off young Pablo formative years in art school, and his time later hanging out with Catalonia fin-de-siècle avant-garde.''),

('1300','2018-08-01 18:00:00','2018-08-01 20:00:00','The National Art Museum of Catalonia, where you can get an excellent overview of Catalan

art spanning from the 12th to the 20th centuries.''),

('1300','2018-08-12 12:00:00','2018-08-12 17:30:00','One of Madrid's heavy hitters, the Reina Sofia specializes in 20th-century Spanish art.

While most Madrid visitors make a beeline for the Prado, this museum is often next on the agenda. The glass addition to the otherwise historic

facade hints at the surprising gems within, including masterpieces focusing on feminism, dreams, the Spanish civil war, and most notably,

Picasso's "Guernica." Rotating exhibitions span the globe and touch upon thought-provoking, conversation-starting topics. This is one of the world's largest museums dedicated

to modern art, quite an impressive feat for a museum that opened in 1992. You will find works from Pablo Picasso, Salvador Dalí, and other bold-faced names.''),

('1201','2023-01-06 12:00:00','2023-01-06 18:00:00','The Colosseum in Rome, Italy, is a large amphitheater that hosted events like gladiatorial games. Design Pics Inc. The Colosseum, also named the Flavian Amphitheater, is a large amphi

theater in Rome. It was built during the reign of the Flavian emperors as a gift to the Roman people.''),

('1201','2023-01-17 11:00:00','2023-01-17 19:00:00','The Bourbon Tunnel (Galleria Borbonica), Tunnel Borbonico or Bourbon Gallery is an ancient underground

passage that was constructed to connect the Royal Palace to military barracks in Naples. Presently the tunnels are open for tours, and contains decades of debris,

including vintage cars and a discarded fascist monument made for Aurelio Padovani,

captain of Bersaglieri during the First World War and founder of the Neapolitan fascist party.''),

('1201','2023-01-23 16:00:00','2023-01-23 19:00:00','The Uffizi Gallery in Florence holds a huge collection of valuable works mostly from the Italian Renaissance.

The building was designed by Giorgio Vasari in 1560 for Cosimo I de Medici, the second Duke of Florence, to house administrative offices and the state archive.

Over the time, the Medici family filled up the rooms with more and more paintings and sculptures from their private collection and eventually the house opened

to the public as an art gallery. The Galleria degli Uffizi boasts paintings by Rembrandt, Michelangelo,

Albrecht Dürer, Titian and Raphael. One room is dedicated to interesting objects collected by the Medici family.''),

('1201','2023-02-10 18:00:00','2023-02-10 20:00:00','The National Museum of Cinema (Museo Nazionale del Cinema) is an Italian motion picture museum, that is

housed in the Mole Antonelliana tower in Turin. The collections includes pre-cinematographic optical devices (magic lanterns), earlier and current film technologies,

stage items from old Italian movies and various memorabilia. Furthermore, there are collections of film posters, stocks, and a library, with 20,000 devices, paintings

and printed artworks, 80,000 pictures, more than 300,000 film posters, around 12,000 movie reels and around 26,000 books related to film.

The National Museum of Cinema and its collection is divided over 5 floors.''),

('1301','2023-02-14 12:00:00','2023-02-14 15:30:00','The Morgan Library & Museum, formerly the Pierpont Morgan Library, is a museum and research

library in the Murray Hill neighborhood of Manhattan in New York City. It is situated at 225 Madison Avenue, between 36th Street to the south and 37th Street to the north. The Morgan Library & Museum is composed of several structures.

The main building was designed by Charles McKim of the firm of McKim, Mead and White, with an annex designed by Benjamin Wistar Morris. A 19th-century Italianate brownstone house at

231 Madison Avenue, built by Isaac Newton Phelps, is also part of the grounds. The museum and library also contains a glass entrance building designed by Renzo Piano and Beyer Blinder Belle.

The main building and its interior is a New York City designated landmark and a National Historic Landmark,

while the house at 231 Madison Avenue is a New York City landmark.''),

('1301','2023-03-02 11:00:00','2023-03-02 19:30:00','Griffith Observatory is an observatory in Los Angeles, California on the south-facing slope of Mount Hollywood in Griffith Park. It commands a view of the Los Angeles Basin including Downtown Los Angeles to the southeast,

Hollywood to the south, and the Pacific Ocean to the southwest. The observatory is a popular tourist attraction with a close view of the Hollywood Sign

and an extensive array of space and science-related displays. It is named after its benefactor, Griffith J. Griffith.

Admission has been free since the observatory opening in 1935');

```
create table IT(  
IT_AT char(10),  
IT_password char(10) DEFAULT 'password' NOT NULL,  
start_date datetime not null,  
end_date datetime, /*Για IT που έχουν φύγει από το πρακτορείο*/  
primary key (IT_AT),  
foreign key (IT_AT) references worker(wrk_AT)  
on delete cascade on update cascade  
);
```

```
CREATE TABLE logg(  
log_AT int(10) NOT NULL AUTO_INCREMENT,  
action1 ENUM('INSERT', 'UPDATE', 'DELETE') NOT NULL,  
table_act ENUM('trip', 'reservation', 'event', 'travel_to',  
'destination') NOT NULL,  
it_lname VARCHAR(20) NOT NULL,  
act_date DATETIME NOT NULL,  
PRIMARY KEY(log_AT)  
);
```

```
create table offers(  
offer_tr_id int(11),  
offer_st_date datetime,
```

```

offer_end_date datetime,
cost float(7,2),
offer_dst_id int(11),
primary key (offer_tr_id,offer_dst_id), /*Warning βάζουμε και το
offer_dst_id ως primary key να μπορούμε να έχουμε πολλά destinations
σε ένα trip ακόμα και να είναι offer trip */
/*foreign key (offer_tr_id) references trip(tr_id)
on delete cascade on update cascade, */
foreign key (offer_dst_id) references destination(dst_id)
on delete cascade on update cascade
);

```

```

create table reservation_offers(
  res_off_name varchar(20),
  res_off_lname varchar(20),
  res_off_id int(12) not null auto_increment,
  res_offer_tr_id int (11), /**/
  deposit_cost float(7,2),
  primary key (res_off_id),
  foreign key (res_offer_tr_id) references offers(offer_tr_id)
  on delete cascade on update cascade
);

```

```

ALTER TABLE reservation_offers AUTO_INCREMENT = 250000;

```

```

insert into IT values
('12345ABCDE',DEFAULT, '2019-08-12 09:00:00',NULL),
('12345ABCDF',DEFAULT, '2018-09-12 10:00:00',NULL),
('12345ABCDG',DEFAULT, '2020-10-12 11:30:00',NULL),
('12345ABCDH',DEFAULT, '2021-11-12 12:00:00',NULL),
('12345ABCDI',DEFAULT, '2022-12-12 09:45:00',NULL),

('12345ABCKL',DEFAULT, '2019-06-28 16:00:00', '2020-09-21 12:30:00'),
('12345ABCMN',DEFAULT, '2020-07-12 18:00:00', '2021-10-18 10:00:00'),
('12345ABCOP',DEFAULT, '2023-01-14 20:30:00', '2023-05-19 15:40:00'),
('12345ABCRS',DEFAULT, '2021-04-16 10:00:00', '2020-04-25 14:00:00'),
('12345ABCTY',DEFAULT, '2020-06-22 14:20:00', '2021-03-23 17:20:00'),

/*Δύο άτομα που δουλεύουν την ίδια χρονική περίοδο για αρκετό
χρόνο*/
('12345ABSDF',DEFAULT, '2021-11-12 12:00:00',NULL),
('12345ABPLO',DEFAULT, '2022-12-12 09:45:00',NULL);

```

```

insert into offers values
('2500', '2024-01-01 00:00:00', '2024-12-31 23:59:59', '470.50', '103'),
('2500', '2024-01-01 00:00:00', '2024-12-31 23:59:59', '470.50', '111'),

('2501', '2024-01-01 00:00:00', '2025-12-31 23:59:59', '1500.50', '99'),

('2502', '2024-04-30 00:00:00', '2025-10-31 23:59:59', '700.00', '105');

```

2^ο Κεφάλαιο (Stored Procedures)

3.1.3 Δημιουργία Procedures

3.1.3.1 Stored procedure στην οποία δίνονται ως είσοδοι τα στοιχεία ενός νέου οδηγού (αριθμός ταυτότητας, όνομα, επώνυμο, μισθός τύπος άδειας, τύπος δρομολογίου και μήνες εμπειρίας). Η procedure θα εντοπίζει το υποκατάστημα με τους λιγότερους οδηγούς και θα εισάγει τον οδηγό ως υπάλληλο του συγκεκριμένου υποκαταστήματος. (θα τον εισάγει στους πίνακες worker και driver)

Αρχικά πριν αναφερθούμε για το συγκεκριμένο Procedure ας παρατηρήσουμε το πίνακα που έχουμε στους **driver** και **worker**.

Πατάμε το συγκεκριμένο command για να έχουμε ένα σαφές πίνακα:

```
SELECT *  
FROM worker  
INNER JOIN driver  
ON worker.wrk_AT = driver.driv_AT;
```

Ο πίνακας που δημιουργείται είναι:

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code	driv_AT	driv_license	driv_route	driv_experience
▶	97C9685C31	Kaylum	Gallagher	35500.50	20000	97C9685C31	D	LOCAL	12
	AB74CA2361	Jerry	Wallace	43000.50	20000	AB74CA2361	C	ABROAD	18
	68B9B9909B	Jackson	Davila	43000.60	20001	68B9B9909B	B	LOCAL	43
	710CC7A849	Thomas	Butler	95500.00	20001	710CC7A849	D	ABROAD	78
	369935A4C4	Corey	Orr	34000.50	20002	369935A4C4	C	ABROAD	32
	B1855A1B76	John	Stewart	50125.00	20002	B1855A1B76	B	LOCAL	95
	0337628780	Nataniel	Shepherd	35000.50	20003	0337628780	A	LOCAL	43
	251A2C4988	Haaris	Lucas	25000.00	20003	251A2C4988	A	LOCAL	35
	72A3AB5128	Earl	Kemp	90500.50	20004	72A3AB5128	A	ABROAD	37

Παρατηρούμε με το κώδικα (wrk_br_code) ότι το υποκατάστημα (branch) 20004 έχει λιγότερους οδηγούς.

Κώδικας:

```
DROP PROCEDURE IF EXISTS getBranchDriverNum;  
DELIMITER $  
CREATE procedure getBranchDriverNum(IN AT_wrk_driver char(10),IN  
wrk_driver_name varchar(20),IN wrk_driver_lname varchar(20),IN  
wrk_driver_salary float(7,2), /*OUT wrk_brancode int(11),*/  
IN driver_AT char(10),IN driver_license enum('A','B','C','D'),IN  
driver_route enum('LOCAL','ABROAD'),IN driver_experience tinyint(4))
```

```
BEGIN  
declare wrk_brancode int(11) ;  
SET @min=(SELECT COUNT(wrk_br_code) FROM worker);  
set @min=wrk_brancode+1;  
  
select wrk_br_code into wrk_brancode from worker where  
AT_wrk_driver=wrk_AT;  
  
select min(wrk_brancode) into @min from (select count(*) wrk_br_code  
from worker group by wrk_br_code) w;
```

```

select * from worker w1
join (select wrk_AT from worker group by wrk_br_code having count(*)
= @min) w2
on w1.wrk_br_code=w2.wrk_br_code;

END$
DELIMITER ;

```

Στην συνέχεια καλούμε το Procedure για να το τεστάρουμε αλλά και να το ενεργοποιήσουμε.

Άρα χρησιμοποιούμε το command:

```

CALL AddNewDriver('1234567890', 'John', 'Curry', 5000.00, 'B',
'LOCAL', 12);

```

Στο ενημερωμένο πίνακα παρατηρούμε ότι έχουμε τον John Curry που μπαίνει στο branch με κώδικα **20004**:

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code	drv_AT	drv_license	drv_route	drv_experience
▶	97C9685C31	Kaylum	Gallagher	35500.50	20000	97C9685C31	D	LOCAL	12
	AB74CA2361	Jerry	Wallace	43000.50	20000	AB74CA2361	C	ABROAD	18
	68B9B9909B	Jackson	Davila	43000.60	20001	68B9B9909B	B	LOCAL	43
	710CC7A849	Thomas	Butler	95500.00	20001	710CC7A849	D	ABROAD	78
	369935A4C4	Corey	Orr	34000.50	20002	369935A4C4	C	ABROAD	32
	B1855A1B76	John	Stewart	50125.00	20002	B1855A1B76	B	LOCAL	95
	0337628780	Nataniel	Shepherd	35000.50	20003	0337628780	A	LOCAL	43
	251A2C4988	Haaris	Lucas	25000.00	20003	251A2C4988	A	LOCAL	35
	1234567890	John	Curry	5000.00	20004	1234567890	B	LOCAL	12
	72A3AB5128	Earl	Kemp	90500.50	20004	72A3AB5128	A	ABROAD	37

Αν ξανά καλέσουμε το Procedure μετά από αυτή την μετατροπή που όλα τα υποκαταστήματα έχουνε ίσο αριθμό οδηγούς θα παρατηρήσουμε ότι το θα γίνει η πρόσθεση στο αρχικό Procedure με το command:

```

CALL AddNewDriver('1234567899', 'Steph', 'Curry', 5000.00, 'B',
'LOCAL', 12);

```

Που έχουμε το πίνακα:

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code	drv_AT	drv_license	drv_route	drv_experience
▶	1234567899	Steph	Curry	5000.00	20000	1234567899	B	LOCAL	12
	97C9685C31	Kaylum	Gallagher	35500.50	20000	97C9685C31	D	LOCAL	12
	AB74CA2361	Jerry	Wallace	43000.50	20000	AB74CA2361	C	ABROAD	18
	68B9B9909B	Jackson	Davila	43000.60	20001	68B9B9909B	B	LOCAL	43
	710CC7A849	Thomas	Butler	95500.00	20001	710CC7A849	D	ABROAD	78
	369935A4C4	Corey	Orr	34000.50	20002	369935A4C4	C	ABROAD	32
	B1855A1B76	John	Stewart	50125.00	20002	B1855A1B76	B	LOCAL	95
	0337628780	Nataniel	Shepherd	35000.50	20003	0337628780	A	LOCAL	43
	251A2C4988	Haaris	Lucas	25000.00	20003	251A2C4988	A	LOCAL	35
	1234567890	John	Curry	5000.00	20004	1234567890	B	LOCAL	12
	72A3AB5128	Earl	Kemp	90500.50	20004	72A3AB5128	A	ABROAD	37

3.1.3.2 Stored procedure στην οποία θα δίνονται ως όρισμα ο κωδικός ενός υποκαταστήματος και δύο ημερομηνίες. Για τα ταξίδια (trip) που διοργανώνονται από το υποκατάστημα του οποίου δόθηκε ο κωδικός, και των οποίων η ημερομηνία αναχώρησης είναι μέσα στο διάστημα που δόθηκε, θα επιστρέφονται τα εξής: Κόστος ταξιδιού (tr_cost), μέγιστες θέσεις maxseat, σύνολο κρατήσεων (reservations), κενές θέσεις (maxseat – σύνολο κρατήσεων), επώνυμο και όνομα οδηγού και ξεναγού, ημερομηνία αναχώρησης και επιστροφής.

Αρχικά, σε αυτό το Procedure βάζουμε τα ορίσματα κώδικας και ημερομηνίες. Στην συνέχεια, ορίζουμε το departure_datetime, trip_id για να καταλάβουμε για ποιο ταξίδι αναφερόμαστε συγκεκριμένα το αριθμό κρατήσεων που θα χρειαστεί να κάνουμε μία εξίσωση για να βρούμε τις διαθέσιμες θέσεις.

(Υπάρχουν επεξηγήσεις και στο κώδικα)

Έπειτα, δημιουργούμε το cursor που θα βρίσκουμε ποιο ταξίδι είναι με το tr_id που θα μας δίνει και το αποτέλεσμα ο πίνακας που δημιουργήσαμε με όλα τα δεδομένα που χρειάζονται.

Ανοίγουμε το cursor με το tr_id στην συνέχεια κάνουμε την συνάρτηση για τις κρατήσεις και τα βάζουμε σε ένα table που θα ενωθεί και με το πίνακα worker.

Στο τέλος χρησιμοποιούμε το * για να μας βγάλει όλα τα δεδομένα του temporary table μαζί με όλα τα δεδομένα της εκφώνησης.

Κώδικας:

```
DROP PROCEDURE IF EXISTS find_trip;
DELIMITER $

CREATE PROCEDURE find_trip(branch_code INT, start_date DATE, end_date DATE)
BEGIN
    DECLARE departure_datetime DATETIME;
    DECLARE trip_id INT;
    DECLARE num_reservations INT;
    DECLARE seats_available TINYINT;
    DECLARE max_seats tinyINT;

    -- Ορίζουμε αυτό το όρισμα για να μπορούμε να καταλαβαίνουμε τις
    -- δύο καταστάσεις που μπορούν να προκύψουν
    DECLARE is_trip_found INT;

    DECLARE trip_cursor CURSOR FOR
    SELECT tr_id FROM trip WHERE tr_br_code = branch_code;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET is_trip_found = 1;

    SET is_trip_found = 0;

    DROP TABLE IF EXISTS temp_table;
    CREATE TABLE temp_table (
        id INT,
        trip_cost FLOAT(7,2),
        max_seats1 INT,
```

```

        reservations1 INT,
        seats_available1 TINYINT,
        guide_first_name1 CHAR(20),
        guide_last_name1 CHAR(20),
        driver_first_name1 CHAR(20),
        driver_last_name1 CHAR(20),
        trip_departure1 DATETIME,
        trip_return1 DATETIME,
        PRIMARY KEY (id)
    );

OPEN trip_cursor;

REPEAT
    FETCH trip_cursor INTO trip_id;
    IF (is_trip_found = 0) THEN
        SELECT tr_departure INTO departure_datetime FROM trip
        WHERE tr_id = trip_id;

        -- Χρησιμοποιούμε την συνάρτηση DATEDIFF που βρίσκει την
        -- διαφορά μεταξύ δύο ημερομηνιών,
        -- Σε αυτή την περίπτωση ανοίγουμε ένα IF και ανάλογα με
        -- την ημερομηνία που θα προσθέσουμε με το CALL,
        -- Θέλουμε να είναι μεσά σε αυτό το διάστημα.
        IF (DATEDIFF(departure_datetime, start_date) > 0 &&
DATEDIFF(departure_datetime, end_date) < 0) THEN
            SELECT COUNT(*) INTO num_reservations FROM
reservation
            where res_tr_id = trip_id;

            select tr_maxseats INTO max_seats FROM trip
            WHERE tr_id = trip_id;

            -- Σύμφωνα με την εκφώνηση
            set seats_available = max_seats - num_reservations;

            INSERT INTO temp_table
            -- Χρειαζόμαστε το tr_id για να μπορούμε να κάνουμε
connection με τον άλλο πίνακα (worker).
            SELECT tr_id, tr_cost, tr_maxseats, num_reservations,
            seats_available, drv.wrk_name, drv.wrk_lname,
            gui.wrk_name, gui.wrk_lname, tr_departure, tr_return
            FROM trip
            inner join worker AS drv ON tr_drv_AT = drv.wrk_AT
            inner join worker AS gui ON tr_gui_AT = gui.wrk_AT
            WHERE tr_id = trip_id;
        END IF;
    END IF;
UNTIL is_trip_found = 1 END REPEAT;

SELECT * FROM temp_table;
END$

DELIMITER ;

```

Για καλύτερη κατανόηση του κώδικα πρώτα ας δείξουμε όλο το πίνακα **trip** με τα data που περιέχει:

	tr_id	tr_departure	tr_return	tr_maxseats	tr_cost	tr_br_code	tr_gui_AT	tr_drv_AT
▶	1200	2019-12-22 21:30:30	2020-01-03 18:00:00	110	349.90	20000	8147862271	97C9685C31
	1201	2023-01-04 09:00:00	2023-02-12 21:15:00	50	320.99	20001	831482CC74	710CC7A849
	1202	2021-02-18 23:00:00	2021-03-02 08:00:00	98	80.75	20002	CA3108032C	B1855A1B76
	1203	2024-05-12 18:00:00	2024-06-02 23:00:00	124	34.00	20003	8B207900AB	251A2C4988
	1204	2022-08-25 07:00:00	2022-09-18 15:00:00	90	560.50	20004	6C5C004CC3	72A3AB5128
	1300	2018-07-30 06:00:30	2018-08-14 05:00:00	89	224.50	20000	8C8166410C	AB74CA2361
	1301	2023-02-13 22:00:00	2023-03-04 15:30:00	105	1240.50	20001	0BB4BC6AB2	68B9B9909B
	1302	2017-06-19 07:00:00	2017-07-02 06:00:00	75	1119.99	20002	43A216CBB4	369935A4C4
	1303	2016-12-22 09:00:00	2017-01-15 12:00:00	60	125.80	20003	15C3A8C1C0	0337628780
	1304	2021-10-23 08:30:00	2021-11-24 18:00:00	122	1200.00	20004	C59C685619	62691A3677

Πίνακας Reservation

	res_tr_id	res_seatnum	res_name	res_lname	res_isadult
▶	1200	1	Lucinda	Roach	ADULT
	1200	9	Edwin	Terrell	MINOR
	1200	10	Marvin	Morgan	ADULT
	1200	15	Vanessa	Proctor	ADULT
	1200	16	Luther	Proctor	MINOR
	1201	2	Haris	Lamb	ADULT
	1201	17	Elspeth	Schneider	MINOR
	1201	39	Georgiana	Franco	ADULT
	1201	50	Denzel	Smith	ADULT
	1300	9	Tara	Case	ADULT
	1300	21	Aleksander	Morris	MINOR
	1300	32	Zach	Duffy	ADULT
	1301	58	Mariam	King	ADULT
	1301	90	Robert	Gross	ADULT

Βλέπουμε και τις κρατήσεις που έχουμε κάνει για να έχουμε μία ποιο σαφή εικόνα και στις θέσεις που είναι διαθέσιμες.

Πρώτη Περίπτωση

Πρώτα ας καλέσουμε το Procedure με άκυρα δεδομένα για να δούμε το αποτέλεσμα.

Για παράδειγμα:

```
call find_trip (17000, '2015-02-25', '2015-04-26');
```

Γράφουμε αυτά τα άκυρα δεδομένα και έχουμε έναν άδειο πίνακα"

	id	trip_cost	max_seats	reservations	seats_available	guide_first_name	guide_last_name	driver_first_name	driver_last_name	trip_departure	trip_return
▶	1200	349.90	110	5	105	Kaylum	Gallagher	Dedan	Hogan	2019-12-22 21:30:30	2020-01-03 18:00:00

Δεύτερη Περίπτωση

Τώρα να βάλουμε δεδομένα που πράγματι υπάρχουν και είναι διαθέσιμα στους πίνακες. Ας βάλουμε το branch 20000 που έχει και reservations.

```
call find_trip (20000, '2019-12-20', '2020-02-05');
```

	id	trip_cost	max_seats1	reservations1	seats_available1	guide_first_name1	guide_last_name1	driver_first_name1	driver_last_name1	trip_departure1	trip_return1
▶	1200	349.90	110	5	105	Kaylum	Gallagher	Dedan	Hogan	2019-12-22 21:30:30	2020-01-03 18:00:00

Που μπορούμε να δούμε ότι υπάρχει σε εκείνες τις ημερομηνίες κάποιο ταξίδι αλλά και τις κρατήσεις. Με αποτέλεσμα να μας βγάλει και τις διαθέσιμες θέσεις.

3.1.3.3 Stored procedure η οποία παίρνει ως όρισμα το όνομα και το επώνυμο ενός υπαλλήλου. Αν είναι διοικητικός τον διαγράφει. Αν ο υπάλληλος είναι διευθυντής υποκαταστήματος τότε εμφανίζεται μήνυμα ότι είναι διευθυντής του υποκαταστήματος και δεν επιτρέπει τη διαγραφή.

Σε αυτό το procedure η βασική προϋπόθεση είναι να δουλέψουμε σε δύο συγκεκριμένους πίνακες. Στον πίνακα **worker** που είναι ο πίνακας που έχουμε τους εργαζόμενους αλλά και όλα τα βασικά στοιχεία (πχ. AM, Όνομα, Επώνυμο κλπ.). Ενώ ο δεύτερος πίνακας είναι ο πίνακας **admin** που αν στο **ENUM** που έχουμε δημιουργήσει είναι στην κατηγορία "ADMINISTRATIVE" τότε σημαίνει ότι κατέχεται και στο πίνακα **manages**. Με αποτέλεσμα να καταλαβαίνουμε ότι είναι διευθυντής.

Θα χρειαστεί να προσέχουμε ότι αυτό το Procedure ισχύει μόνο για το διοικητικό τμήμα. Δηλαδή δεν εξετάζουμε τις κατηγορίες όπως οδηγός, ξεναγός κλπ. Μόνο το πίνακα **admin**.

Κώδικας:

```
DROP PROCEDURE IF EXISTS deleteWorker;
DELIMITER $

CREATE PROCEDURE deleteWorker(worker_name CHAR(10), worker_lname
CHAR(10))
BEGIN
    DECLARE admin_type ENUM('LOGISTICS', 'ADMINISTRATIVE',
'ACCOUNTING');
    DECLARE admin_AT CHAR(10);

    SELECT adm_type, adm_AT INTO admin_type, admin_AT
    FROM admin
    INNER JOIN worker ON adm_AT = wrk_AT
    WHERE worker_name = wrk_name AND worker_lname = wrk_lname;

    IF (admin_type != 'ADMINISTRATIVE' AND admin_type IS NOT NULL)
    THEN
        SELECT ('Ο εργαζόμενος διαγράφηκε με επιτυχία') as message;

        DELETE FROM admin WHERE adm_AT = admin_type;
        DELETE FROM worker WHERE wrk_AT = admin_AT;

    ELSEIF (admin_type IS NULL) THEN
        SELECT ('Ο εργαζόμενος δεν είναι στον πίνακα "ADMIN"') as
message;

    ELSE
        SELECT ('Ο εργαζόμενος είναι διευθυντής υποκαταστήματος, δεν
μπορεί να γίνει η διαγραφή') as message;
    END IF;
END$

DELIMITER ;
```


Ας εξετάσουμε το Procedure που έχουμε δημιουργήσει.

Αρχικά να δώσουμε όλο το πίνακα worker:

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code
►	0337628780	Nataniel	Shepherd	35000.50	20003
	0BB4BC6AB2	Claire	Coleman	60000.50	20001
	15C3A8C1C0	Andreas	Potts	47500.50	20003
	197B244051	Arran	Bender	95500.20	20003
	2161C52254	Rose	Cruz	88000.40	20004
	251A2C4988	Haaris	Lucas	25000.00	20003
	26873C4924	Stephanie	Park	30500.50	20001
	369935A4C4	Corey	Orr	34000.50	20002
	4399691360	Scarlet	Chaney	45000.50	20002
	43A216CBB4	Heidi	Scott	55000.00	20002
	4C045A649A	Olivia	Camacho	38000.60	20003
	5258C7B8C6	Elodie	Carlson	87000.50	20002
	621B027416	Caoimhe	House	25000.00	20000
	68B9B9909B	Jackson	Davila	43000.60	20001
	6C5C004CC3	Marco	Hart	60000.40	20004
	6C738794A1	Lucia	Bray	27000.50	20001
	710CC7A849	Thomas	Butler	95500.00	20001
	72A3AB5128	Earl	Kemp	90500.50	20004
	737787B8B3	Samson	Hampton	78500.50	20000
	745066107B	Lilly-Rose	Wilkerson	98000.50	20002
	757006C2BB	Ellie-Mae	Barr	30500.20	20003
	8147862271	Declan	Hogan	20000.50	20000
	820C85C87C	Kate	Keith	23500.00	20001
	831482CC74	Jimmy	Humphrey	76000.50	20001
	88244C2507	Gerard	Mccoy	55500.50	20004
	8B207900AB	Macie	Cobb	34000.00	20003
	8C8166410C	Dennis	Branch	40000.40	20000
	93C4BBCB6C	Elise	Meyer	40000.40	20004
	97C9685C31	Kaylum	Gallagher	35500.50	20000
	98872BAA72	Sandra	Koch	70500.50	20000
	AB74CA2361	Jerry	Wallace	43000.50	20000
	B1855A1B76	John	Stewart	50125.00	20002
	C59C685619	Gunilla	Bowen	45000.50	20004
	CA3108032C	Anoushka	Bridges	63125.00	20002

Στην συνέχεια να δούμε και τον πίνακα `admin` που έχουμε αλλά ποιο συγκεκριμένα τις κατηγορίες (**`adm_type`**) που έχουν αυτοί οι εργαζόμενοι.

	adm_AT	adm_type	adm_diploma
▶	197B244051	ADMINISTRATIVE	Bachelor of Business Administration Degree Cap...
	2161C52254	ACCOUNTING	Management: Accounting Cazenovia College Ca...
	26873C4924	LOGISTICS	Bachelor in International Logistics Management,...
	4399691360	LOGISTICS	Bachelor of Commerce in Operations Manageme...
	4C045A649A	ACCOUNTING	Bachelor in Accounting,Carroll University
	5258C7B8C6	ADMINISTRATIVE	B.A./B.S. in Transportation and Logistics Manag...
	621B027416	ACCOUNTING	BA (Hons) Business Management (Accounting a...
	6C738794A1	ACCOUNTING	Bachelor of Science - Accounting Eastern Univer...
	737787B8B3	ADMINISTRATIVE	B.S.B.A. Management Nova Southeastern Univ...
	745066107B	ACCOUNTING	Bachelor of Business - Major in Accounting Melb...
	757006C2BB	LOGISTICS	Bachelor of Arts (Honours) Business Logistics an...
	820C85C87C	ADMINISTRATIVE	B.A. Public Administration University of Tenness...
	88244C2507	ADMINISTRATIVE	BSC in Business Administration Worcester State ...
	93C4BBCB6C	LOGISTICS	BSc (Hons) in Finance, Operations Research, M...
	98872BAA72	LOGISTICS	Bachelor in Logistics Engineering Fontys Univers...
*	NULL	NULL	NULL

Προσοχή !!

Σε κάθε περίπτωση έχουμε ξανατρέξει τον κώδικα. Δηλαδή αν σε μία περίπτωση διαγράφεται κάποιος στην άλλη περίπτωση αυτός ο εργαζόμενος παραμένει αφού δεν είναι η δική του περίπτωση επειδή έχουμε ξανατρέξει το κώδικα για να δείξουμε μία άλλη περίπτωση.

1^η Περίπτωση:

Ας διαλέξουμε έναν υπάλληλο που δεν είναι **manager** δηλαδή δεν ανήκει στην κατηγορία "ADMINISTRATIVE" αλλά είναι διοικητικός δηλαδή ανήκει στον πίνακα **admin**.

Ας πάρουμε τον εργαζόμενο με `adm_AT '2161C52254'` που ανήκει στην κατηγορία "ACCOUNTING".

Από το πίνακα **worker** μπορούμε να δούμε ότι η εργαζόμενος έχει το όνομα και επίθετο **Rose Cruz**.

Καλούμε το Procedure με:

```
call deleteWorker('Rose','Cruz')
```

Που βλέπουμε το μήνυμα:

	message
▶	Ο εργαζόμενος διαγράφηκε με επιτυχία

Για να είμαστε σίγουροι θα χρειαστεί να δούμε αν υπάρχει το ονοματεπώνυμο και ΑΜ που έχουμε διαγράψει από τους πίνακες:

```
select * FROM worker WHERE wrk_name LIKE 'Rose'
```

Έχουμε το αποτέλεσμα:

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code
*	NULL	NULL	NULL	NULL	NULL

Το ίδιο κάνουμε και για το πίνακα admin με το κώδικα:

```
select * FROM admin WHERE adm_type LIKE 'ACCOUNT%'
```

Που παρατηρούμε ότι στην κατηγορία “ACCOUNTING” δεν έχουμε το adm_AT '2161C52254'. Άρα έχει γίνει η διαγραφή.

	adm_AT	adm_type	adm_diploma
▶	4C045A649A	ACCOUNTING	Bachelor in Accounting,Carroll University
	621B027416	ACCOUNTING	BA (Hons) Business Management (Accounting a...
	6C738794A1	ACCOUNTING	Bachelor of Science - Accounting Eastern Univer...
	745066107B	ACCOUNTING	Bachelor of Business - Major in Accounting Melb...

2^η Περίπτωση:

Σε αυτή την περίπτωση θα εξετάσουμε την διαγραφή ενός υπάλληλου που δεν ανήκει στο διοικητικό τμήμα (admin) και άρα δεν είναι και manager. Μπορεί να είναι οδηγός, ξεναγός κλπ.

Ας πάρουμε το παράδειγμα ότι είναι driver.

	drv_AT	drv_license	drv_route	drv_experience
▶	0337628780	A	LOCAL	43
	251A2C4988	A	LOCAL	35
	369935A4C4	C	ABROAD	32
	68B9B9909B	B	LOCAL	43
	710CC7A849	D	ABROAD	78
	72A3AB5128	A	ABROAD	37
	97C9685C31	D	LOCAL	12
	AB74CA2361	C	ABROAD	18
	B1855A1B76	B	LOCAL	95
*	NULL	NULL	NULL	NULL

Βλέπουμε τα **drv_AT** (Αριθμός Ταυτότητας) των **driver** και διαλέγουμε το **driver** με drv_AT '0337628780'.

Βρίσκουμε το driver από το πίνακα worker και βλέπουμε ότι λέγεται **Nataniel Sheperd**

Καλούμε το Procedure που έχουμε δημιουργήσει:

```
call deleteWorker('Nataniel','Sheperd');
```

Που βλέπουμε το μήνυμα ότι δεν ανήκει ο συγκεκριμένος στον πίνακα **"ADMIN"**

message
▶ Ο εργαζόμενος δεν είναι στον πίνακα "ADMIN"

Που βλέπουμε ότι δεν έχει γίνει η διαγραφή και ότι παραμένει ο εργαζόμενος.

Με κώδικα: `select * FROM worker WHERE wrk_name LIKE 'Natan%'`

wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code
0337628780	Nataniel	Shepherd	35000.50	20003

Που μπορούμε να δούμε ότι είναι και στο πίνακα driver κανονικά από το Αριθμό Ταυτότητας **'0337628780'**.

drv_AT	drv_license	drv_route	drv_experience
0337628780	A	LOCAL	43
251A2C4988	A	LOCAL	35
369935A4C4	C	ABROAD	32
68B9B9909B	B	LOCAL	43
710CC7A849	D	ABROAD	78
72A3A85128	A	ABROAD	37
97C9685C31	D	LOCAL	12
AB74CA2361	C	ABROAD	18
B1855A1B76	B	LOCAL	95

3^η Περίπτωση:

Στην τελευταία περίπτωση θα εξετάσουμε το γεγονός ότι εργαζόμενος είναι διοικητικός και ανήκει στην υποκατηγορία **"ADMINISTRATIVE"**. Που σημαίνει ότι είναι manager ενός υποκαταστήματος και θα πρέπει να μην αποδέχεται η διαγραφή του.

Για αυτή την εξέταση θα διαλέξουμε τον εργαζόμενο με αριθμό ταυτότητας **'5258C7B8C6'**. Που είναι στο πίνακα **admin** και ανήκει στην κατηγορία **"ADMINISTRATIVE"**.

Από το πίνακα **worker** βλέπουμε ότι λέγεται **Elodie Carlson**.

Ας καλέσουμε το Procedure για την 3^η περίπτωση:

Με κώδικα: `CALL deleteWorker('Elodie','Carlson');`

Βλέπουμε ότι μας βγάζει το παρακάτω μήνυμα και δεν γίνεται η διαγραφή.

message
▶ Ο εργαζόμενος είναι διευθυντής υποκαταστήματος...
Ο εργαζόμενος είναι διευθυντής υποκαταστήματος, δεν μπορεί να γίνει η διαγραφή

Ας κοιτάξουμε και στους πίνακες αν έχει γίνει η διαγραφή.

Με κώδικα: `select * FROM worker WHERE wrk_name LIKE 'Elod%'`

Βλέπουμε ότι στο πίνακα worker δεν έχει γίνει η διαγραφή και παραμένει ως **worker** η Elodie.

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code
►	5258C7B8C6	Elodie	Carlson	87000.50	20002

Δεν έχει γίνει καμία διαγραφή και στο πίνακα admin που το παρατηρούμε ότι έχουμε το ίδιο AM ακόμα στο πίνακα admin με αυτό που έχει η Elodie.

	adm_AT	adm_type	adm_diploma
►	197B244051	ADMINISTRATIVE	Bachelor of Business Administration Degree Cap...
	2161C52254	ACCOUNTING	Management: Accounting Cazenovia College Ca...
	26873C4924	LOGISTICS	Bachelor in International Logistics Management,...
	4399691360	LOGISTICS	Bachelor of Commerce in Operations Manageme...
	4C045A649A	ACCOUNTING	Bachelor in Accounting,Carroll University
	5258C7B8C6	ADMINISTRATIVE	B.A./B.S. in Transportation and Logistics Manag...
	621B027416	ACCOUNTING	BA (Hons) Business Management (Accounting a...
	6C738794A1	ACCOUNTING	Bachelor of Science - Accounting Eastern Univer...
	737787B8B3	ADMINISTRATIVE	B.S.B.A. Management Nova Southeastern Univ...
	745066107B	ACCOUNTING	Bachelor of Business - Major in Accounting Melb...
	757006C2BB	LOGISTICS	Bachelor of Arts (Honours) Business Logistics an...
	820C85C87C	ADMINISTRATIVE	B.A. Public Administration University of Tennes...
	88244C2507	ADMINISTRATIVE	BSC in Business Administration Worcester State ...
	93C4BB6B6C	LOGISTICS	BSc (Hons) in Finance, Operations Research, M...
	98872BAA72	LOGISTICS	Bachelor in Logistics Engineering Fontys Univers...

3.1.3.4 Σε συνέχεια της απαίτησης 3.1.2.3, λόγω του μεγάλου πλήθους εγγραφών ,θα πρέπει να δημιουργήσετε επιπλέον ευρετήριο στον πίνακα reservation_offers. **Για κάθε μια από τις stored procedures θα επιλέξετε το κατάλληλο ευρετήριο και να αιτιολογήσετε την επιλογή σας, έτσι ώστε να αντιμετωπίζεται αποδοτικά τα ζητούμενα παρακάτω. Θα καταγράψετε τους χρόνους για κάθε εκτέλεση με και χωρίς ευρετήριο, θα επισυνάψετε κατάλληλο snapshot σχολιάζοντας το.**

α)**Stored procedure** στην οποία δίνονται ως παράμετροι εισόδου δύο τιμές (πχ 50, 200) και επιστρέφει τους πελάτες (επώνυμο, όνομα) που έκαναν κρατήσεις σε προσφορές ταξιδιών και πλήρωσαν για προκαταβολή, ποσό ανάμεσα στις δύο τιμές.

Απάντηση:

Σύμφωνα με την εκφώνηση που έχουμε αρχικά θα χρειαστούμε να δημιουργήσουμε ένα **Index** που θα εντοπίζεται στο Procedure. Βέβαια, το Index που δημιουργούμε βοηθά και δίνει κάποια **hints** για την εύρεση των δεδομένων που φτιάχνουμε και να μην ασχολείται με άλλα δεδομένα.

Επίσης, για να χρησιμοποιήσουμε το Procedure χωρίς το Index θα πρέπει απλά να βγάλουμε το αναφερόμενο κώδικα για το Index

Σχετικά με το **Procedure**, αρχικά βάζουμε 2 **Float** τιμές ως είσοδο που θα τις χρησιμοποιήσουμε στην συνέχεια ως τιμές δηλαδή θα είναι τα δεδομένα που θα μας περιορίζουν ένα διάστημα.
Όπως, έχουμε αναφέρει και στο κώδικα βάζουμε τις τιμές που θα επεξεργαστούμε και θα κάνουμε ουσιαστικά ένα matching με δεδομένα των πινάκων.

Ανοίγουμε ένα cursor αρχικά για να δούμε αν υπάρχουν δεδομένα στο πίνακα για ασφάλεια αλλά και να μας βγάλει **error** σε περίπτωση που δεν έχουμε δεδομένα. Αφού κάνουμε set ότι έχουμε δεδομένα συνεχίζουμε το κώδικα. Στην συνέχεια θα χρησιμοποιήσουμε το συγκεκριμένο cursor που θα φέρουμε και άλλα δεδομένα για την προσωρινή μνήμη.

Δημιουργούμε έναν νέο προσωρινό πίνακα στο **Procedure**, ανοίγουμε και το Cursor που έχουμε δημιουργήσει παραπάνω που το χρησιμοποιούμε σαν Loop. Τα variables που έχουμε δημιουργήσει ποιο πριν τους ενώνουμε με τους πίνακες του κώδικα. Τέλος, εισάγουμε τα δεδομένα στο προσωρινό πίνακα όπως μας τα ζητάει η εκφώνηση αλλά και το Index.

Κώδικας:

```
-- Το INDEX που δημιουργούμε για το PROCEDURE
CREATE INDEX depositcost_index
ON reservation_offers (deposit_cost);

DROP PROCEDURE IF EXISTS reservation_pay;

DELIMITER $

CREATE PROCEDURE reservation_pay(deposit1 FLOAT(7,2), deposit2
FLOAT(7,2))
BEGIN
    -- Βάζουμε τα variables που θέλουμε να επεξεργαστούμε
    DECLARE depositpay FLOAT(7,2);
    DECLARE reservation_am INT(11);
    DECLARE not_available INT;

    -- Ανοίγουμε Cursor που θα βάλουμε ισότητα μαζί με reservation_am
    DECLARE precursor CURSOR FOR
        SELECT res_off_id FROM reservation_offers;

    -- Σε περίπτωση που δεν βρίσκει το Procedure
    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET not_available = 1;

    SET not_available = 0;

    DROP TABLE IF EXISTS payment;

    -- Δημιουργούμε πίνακα
    CREATE TABLE payment (
        fname varchar(20),
```

```

        lname varchar(20),
        roid INT(11),
        PRIMARY KEY(roid)
    );

-- Ανοίγουμε Cursor
OPEN precursor;

-- Έναρξη loop για λήψη και επεξεργασία δεδομένων
REPEAT
    -- Φέρνουμε το reservation_am από τα Declared
    FETCH precursor INTO reservation_am;

    -- Έλεγχος αν έχει λάβει
    IF(not_available = 0) THEN
        -- Λάβουμε προκαταβολή για την τρέχουσα κράτηση
        SELECT deposit_cost INTO depositpay FROM
reservation_offers
        WHERE res_off_id = reservation_am;

        -- Εισάγουμε δεδομένα στον πίνακα βάσει συνθηκών
        INSERT INTO payment
        SELECT res_off_name, res_off_lname, res_off_id FROM
reservation_offers
        USE INDEX (depositcost_index)
        WHERE res_off_id = reservation_am
            && deposit_cost >= deposit1
            && deposit_cost <= deposit2;
    END IF;

    -- Συνεχίζουμε το loop μέχρι να μην βρεθούν άλλες σειρές
    UNTIL not_available = 1 END REPEAT;

    -- Επιλέγουμε και εμφανίζουμε τα ονόματα και τα επώνυμα από τον
    πίνακα
    SELECT fname AS FName, lname AS LastName FROM payment;

END$

-- Κάνουμε Reset το Delimiter
DELIMITER ;

```

Για να καλέσουμε το Procedure:

```
call reservation_pay(95.00,97.00);
```

Ως αποτέλεσμα έχουμε:

	FName	LastName
▶	Aliyah	Holmes
	Viviana	Mooney
	Janet	Cameron
	Kyan	Tapia
	Sophie	Fitzpatrick
	German	Oconnor
	Emilio	Valdez
	Angela	Hays
	Jaquan	Hancock
	Katherine	Brewer
	Charlee	Torres
	Kiana	Bridges
	Kianna	King
	Kael	Glass
	Erika	Coleman
	Emmy	Francis
	Elliott	Flowers
	Tobias	Davenport
	Jaeden	Wong
	Maurice	Huff

Όλα τα ονόματα που έχουν κράτηση στις τιμές που έχουμε ορίσει. Που αυτή η λίστα συνεχίζεται αλλά θα ήταν αδύνατο να το βάλουμε σε screenshot.

Με την χρήση του **Index**

⚠	131	21:29:32	CREATE PROCEDURE reservation_pay(deposit1 FLOAT(7,2), ...	0 row(s) affected, 7 warning(s): 1681 Specifying number of digits ...	0.016 sec
✓	132	21:29:32	call reservation_pay(95.00,97.00)	1178 row(s) returned	26.656 sec / 0.000 sec

Χωρίς Χρήση του **Index**

⚠	175	21:31:13	CREATE PROCEDURE reservation_pay(deposit1 FLOAT(7,2), ...	0 row(s) affected, 7 warning(s): 1681 Specifying number of digits ...	0.015 sec
✓	176	21:31:13	call reservation_pay(95.00,97.00)	1178 row(s) returned	12.219 sec / 0.000 sec

Παρατηρούμε ότι δεν υπάρχει μεγάλη διαφορά στην χρήση Index που μας βγάζει τα δεδομένα σε αρκετό γρήγορο χρόνο και στις 2 περιπτώσεις.

β) Stored procedure στην οποία δίνεται ως παράμετρος εισόδου το επώνυμο ενός πελάτη και επιστρέφει τα ονόματα και επώνυμα των πελατών με το επώνυμο αυτό και η προσφορά ταξιδιού στην οποία έχει γίνει εγγραφή. Σε περίπτωση που υπάρχουν πάνω από ένας επιστρέφει το πλήθος των πελατών με το όνομα αυτό, ανά προσφορά ταξιδιού.

Απάντηση:

Αρχικά, βάζουμε το επώνυμο ως είσοδο στο **Procedure** όπως αναφέρεται και η εκφώνηση.

Έχουμε δύο declare που το pass_id θα το ενώσουμε με τα δεδομένα **res_off_id** και not_available που έχει την ίδια χρήση με το α) ερώτημα.

Στην συνέχεια ανοίγουμε το cursor που θα χρησιμοποιηθεί ως μνήμη για να βάλουμε τα δεδομένα που θα χρειαστούν από το αρχικό κώδικα αλλά και το **lname** ώστε να μπορούμε να τα ταιριάσουμε και με το νέο πίνακα **new_res_off** που είναι για το Procedure και γίνεται η αποθήκευση.

Στο τέλος, βάζουμε τα data του reservation_offers στο νέο μας πίνακα και γίνεται το matching με τα data που ασχολούμαστε παρακάτω.

Όπως και στην α) ερώτηση έχουμε χρησιμοποιήσει το **INDEX** για την σύγκριση,

Κώδικας:

```
CREATE INDEX passengerfind_index
ON reservation_offers (res_off_lname);

DROP PROCEDURE IF EXISTS find_passenger;

DELIMITER $

CREATE PROCEDURE find_passenger (lname VARCHAR(20))
BEGIN
    DECLARE pass_id INT(12);
    DECLARE not_available INT;

    DECLARE passcursor CURSOR FOR
        SELECT res_off_id
        FROM reservation_offers
        WHERE res_off_lname = lname;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET not_available = 1;

    SET not_available = 0;

    DROP TABLE IF EXISTS new_res_off;

    CREATE TABLE new_res_off (
        noff_id INT(11),
        reserv_id INT(12),
        nfirst_name VARCHAR(20),
        nlast_name VARCHAR(20),
        PRIMARY KEY (reserv_id)
    );

    OPEN passcursor;
```

```

REPEAT
    FETCH passcursor INTO pass_id;

    IF (not_available = 0) THEN
        INSERT INTO new_res_off
        SELECT res_offer_tr_id, res_off_id, res_off_name,
res_off_lname
        FROM reservation_offers
        USE INDEX (passengerfind_index)
        WHERE res_off_lname = lname AND res_off_id = pass_id;
    END IF;
UNTIL (not_available = 1)
END REPEAT;

SELECT noff_id AS Offer_ID, nfirst_name AS First_Name, nlast_name
AS Last_Name
FROM new_res_off
ORDER BY noff_id;
END$

DELIMITER ;

```

Καλούμε το Procedure με το επίθετο “Holmes” :

```
call find_passenger ('Holmes');
```

	Offer_ID	First_Name	Last_Name
	2500	Deon	Holmes
	2500	Izabelle	Holmes
	2500	Nicole	Holmes
	2500	Janet	Holmes
	2500	Shyann	Holmes
	2500	Jaylee	Holmes
	2500	Jagger	Holmes
	2500	Fernanda	Holmes
	2500	Aliyah	Holmes
	2500	Simeon	Holmes
	2500	Thalia	Holmes
	2500	Deon	Holmes
	2500	Izabelle	Holmes
	2500	Nicole	Holmes
	2500	Janet	Holmes
	2500	Shyann	Holmes
	2500	Jaylee	Holmes
	2500	Jagger	Holmes
	2500	Fernanda	Holmes
	2501	Aliyah	Holmes

Με την χρήση του **Index**

	140	14:30:46	CREATE PROCEDURE find_passenger (@name VARCHAR(20)) ...	0 row(s) affected, 3 warning(s): 1681 Integer display width is depr...	0.000 sec
	141	14:30:46	call find_passenger ('Holmes')	66 row(s) returned	0.391 sec / 0.000 sec

Χωρίς Χρήση του **Index**

	187	14:33:01	CREATE PROCEDURE find_passenger (@name VARCHAR(20)) ...	0 row(s) affected, 3 warning(s): 1681 Integer display width is depr...	0.016 sec
	188	14:33:01	call find_passenger ('Holmes')	66 row(s) returned	0.484 sec / 0.000 sec

Παρατηρούμε ότι δεν υπάρχει μεγάλη διαφορά στην χρήση Index που μας βγάζει τα δεδομένα σε αρκετό γρήγορο χρόνο και στις 2 περιπτώσεις.

Βλέπουμε ότι υπάρχει διαφορά στα 2 **α)** και **β)** ερωτήματα με τα **INDEX** που ο λόγος θα είναι ότι στο πρώτο **INDEX** παρόλο που έχουμε βάλει ένα σωστό **HINT** θα χρειαστεί να κάνουμε μία σύγκριση που το **HINT** δεν βοηθά άμεσα αλλά έμμεσα. Ενώ στο **β)** βοηθά άμεσα με το επίθετο και γίνεται γρηγορότερο με την χρήση του **INDEX**. Εν τέλη, δεν υπάρχει μία διαφορά που να είναι σοβαρή.

3^ο Κεφάλαιο (Triggers)

3.1.4.1. **Trigger** που θα ενημερώνουν το σχετικό πίνακα καταγραφής **ενεργειών (log)** για κάθε ενέργεια εισαγωγής, ενημέρωσης ή διαγραφής στους πίνακες **trip, reservation, event, travel_to, destination**, καθώς και το ποιος την εκτέλεσε.

Απάντηση: Σε αυτό το Trigger αρχικά θα χρειαστεί να κάνουμε ένα νέο trigger σε κάθε ενέργεια που κάνει ο IT (insert,update,delete) που θα είναι για κάθε πίνακα με αποτέλεσμα να έχουμε 15 triggers για κάθε ενέργεια.

Δεν υπάρχει καμία αλλαγή στην φιλοσοφία κάθε trigger, δηλαδή έχουμε την ίδια λογική για κάθε ένα. Η μόνη διαφορά είναι τα ονόματα των trigger και η ενέργεια που υλοποιείται.

Αρχικά κάνουμε ένα declare για το επίθετο του IT που το ενώνουμε πρώτα στο επίθετο του πίνακα worker και στην συνέχεια με INNER JOIN στο πίνακα IT με το AT του IT για να ξέρουμε ποιος IT κάνει την ενέργεια.

Τέλος, κάνουμε Insert στο πίνακα logg γράφοντας την ενέργεια που υλοποιούμε και το πίνακα που γίνεται η ενέργεια.

Χρησιμοποιούμε την συνάρτηση now() που μας δίνει την ημερομηνία που γίνεται η πράξη.

Κώδικας για το Reservation

```
-- Trigger for INSERT on reservation table
DROP TRIGGER IF EXISTS Reservation_Insert_Trigger;
DELIMITER $
CREATE TRIGGER Reservation_Insert_Trigger
AFTER INSERT ON reservation
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('INSERT', 'reservation', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for UPDATE on reservation table
DROP TRIGGER IF EXISTS Reservation_Update_Trigger;
DELIMITER $
CREATE TRIGGER Reservation_Update_Trigger
AFTER UPDATE ON reservation
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;
```

```

-- -- Insert σε πίνακα logg
INSERT INTO logg (action1, table_act, it_lname, act_date)
VALUES ('UPDATE', 'reservation', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for DELETE on reservation table
DROP TRIGGER IF EXISTS Reservation_Delete_Trigger;
DELIMITER $
CREATE TRIGGER Reservation_Delete_Trigger
AFTER DELETE ON reservation
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('DELETE', 'reservation', worker_lastname, NOW());
END $
DELIMITER ;

```

Κώδικας για το Trip

```

-- Trigger for INSERT on trip table
DROP TRIGGER IF EXISTS Trip_Insert_Trigger;
DELIMITER $
CREATE TRIGGER Trip_Insert_Trigger
AFTER INSERT ON trip
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('INSERT', 'trip', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for UPDATE on trip table
DROP TRIGGER IF EXISTS Trip_Update_Trigger;
DELIMITER $
CREATE TRIGGER Trip_Update_Trigger
AFTER UPDATE ON trip
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

```

```

SELECT wrk_lname INTO worker_lastname
FROM worker w
INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

-- -- Insert σε πίνακα logg
INSERT INTO logg (action1, table_act, it_lname, act_date)
VALUES ('UPDATE', 'trip', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for DELETE on trip table
DROP TRIGGER IF EXISTS Trip_Delete_Trigger;
DELIMITER $
CREATE TRIGGER Trip_Delete_Trigger
AFTER DELETE ON trip
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('DELETE', 'trip', worker_lastname, NOW());
END $
DELIMITER ;

```

Κώδικας για το Travel to

```

-- Trigger for INSERT on travel_to table
DROP TRIGGER IF EXISTS Travel_to_Insert_Trigger;
DELIMITER $
CREATE TRIGGER Travel_to_Insert_Trigger
AFTER INSERT ON travel_to
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('INSERT', 'travel_to', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for UPDATE on travel_to table
DROP TRIGGER IF EXISTS Travel_to_Update_Trigger;

```

```

DELIMITER $
CREATE TRIGGER Travel_to_Update_Trigger
AFTER UPDATE ON travel_to
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('UPDATE', 'travel_to', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for DELETE on travel_to table
DROP TRIGGER IF EXISTS Travel_to_Delete_Trigger;
DELIMITER $
CREATE TRIGGER Travel_to_Delete_Trigger
AFTER DELETE ON travel_to
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('DELETE', 'travel_to', worker_lastname, NOW());
END $
DELIMITER ;

```

Κώδικας για το Event

```

-- Trigger for INSERT on event table
DROP TRIGGER IF EXISTS Event_Insert_Trigger;
DELIMITER $
CREATE TRIGGER Event_Insert_Trigger
AFTER INSERT ON event
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)

```

```

        VALUES ('INSERT', 'event', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for UPDATE on event table
DROP TRIGGER IF EXISTS Event_Update_Trigger;
DELIMITER $
CREATE TRIGGER Event_Update_Trigger
AFTER UPDATE ON event
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('UPDATE', 'event', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for DELETE on event table
DROP TRIGGER IF EXISTS Event_Delete_Trigger;
DELIMITER $
CREATE TRIGGER Event_Delete_Trigger
AFTER DELETE ON event
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('DELETE', 'event', worker_lastname, NOW());
END $
DELIMITER ;

```

Κώδικας για το Destination

```

-- Trigger for INSERT on destination table
DROP TRIGGER IF EXISTS Destination_Insert_Trigger;
DELIMITER $
CREATE TRIGGER Destination_Insert_Trigger
AFTER INSERT ON destination
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname

```



```

        FROM worker w
        INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

-- -- Insert σε πίνακα logg
INSERT INTO logg (action1, table_act, it_lname, act_date)
VALUES ('INSERT', 'destination', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for UPDATE on destination table
DROP TRIGGER IF EXISTS Destination_Update_Trigger;
DELIMITER $
CREATE TRIGGER Destination_Update_Trigger
AFTER UPDATE ON destination
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('UPDATE', 'destination', worker_lastname, NOW());
END $
DELIMITER ;

-- Trigger for DELETE on destination table
DROP TRIGGER IF EXISTS Destination_Delete_Trigger;
DELIMITER $
CREATE TRIGGER Destination_Delete_Trigger
AFTER DELETE ON destination
FOR EACH ROW
BEGIN
    DECLARE worker_lastname VARCHAR(20);

    SELECT wrk_lname INTO worker_lastname
    FROM worker w
    INNER JOIN IT it ON w.wrk_AT = it.IT_AT WHERE it.IT_AT =
'12345ABCDG' limit 1;

    -- -- Insert σε πίνακα logg
    INSERT INTO logg (action1, table_act, it_lname, act_date)
    VALUES ('DELETE', 'destination', worker_lastname, NOW());
END $
DELIMITER ;

```

Εξέταση Κώδικα

Για να εξετάσουμε το κώδικα των **Trigger**, ας κάνουμε από ένα παράδειγμα σε **UPDATE,DELETE** και **INSERT** που θα γίνουν εγγραφή στο πίνακα **logg**.

Στα παραδείγματα που υλοποιούμε έχουμε βάλει τον ίδιο IT για τους πίνακες reservation και έναν διαφορετικό για το DELETE του travel_to.

Πρώτα ας τυπώσουμε τους πίνακες πριν την υλοποίηση.

Πίνακας travel_to

	to_tr_id	to_dst_id	to_arrival	to_departure
▶	1200	100	2019-12-22 21:30:30	2020-12-27 08:00:00
	1200	101	2019-12-27 17:30:30	2020-01-03 18:00:00
	1200	102	2019-12-27 18:30:30	2020-01-03 19:00:00
	1200	106	2019-12-27 16:30:30	2020-01-03 18:00:00
	1200	107	2019-12-27 15:30:30	2020-01-03 18:00:00
	1201	105	2023-01-04 09:00:00	2023-02-12 21:15:00
	1300	104	2018-07-30 06:00:30	2018-08-07 15:00:00
	1300	111	2018-08-07 21:30:30	2018-08-14 05:00:00
	1301	106	2023-03-01 23:30:00	2023-03-04 15:30:00
	1301	107	2023-02-13 22:00:00	2023-02-27 08:30:00
*	NULL	NULL	NULL	NULL

Πίνακας reservation

	res_tr_id	res_seatnum	res_name	res_lname	res_isadult
▶	1200	1	Lucinda	Roach	ADULT
	1200	9	Edwin	Terrell	MINOR
	1200	10	Marvin	Morgan	ADULT
	1200	15	Vanessa	Proctor	ADULT
	1200	16	Luther	Proctor	MINOR
	1201	2	Haris	Lamb	ADULT
	1201	17	Elsbeth	Schneider	MINOR
	1201	39	Georgiana	Franco	ADULT
	1201	50	Denzel	Smith	ADULT
	1300	9	Tara	Case	ADULT
	1300	21	Aleksander	Morris	MINOR
	1300	32	Zach	Duffy	ADULT
	1301	58	Mariam	King	ADULT
	1301	90	Robert	Gross	ADULT
*	NULL	NULL	NULL	NULL	NULL

Στην συνέχεια κάνουμε τις αλλαγές στους πίνακες:

Για το **UPDATE**

```
SET SQL_SAFE_UPDATES = 0;  
DELETE FROM reservation  
WHERE res_name = 'Zach';  
SET SQL_SAFE_UPDATES = 1;
```

Για το **DELETE**

```
DELETE FROM travel_to  
WHERE to_dst_id= '100';
```

Για το **INSERT**

```
INSERT INTO travel_to  
VALUES ('1200','99','2019-12-28 17:25:30','2020-12-28 17:25:30');
```

Για το **UPDATE**

```
UPDATE reservation  
SET res_name = 'Giannis'  
WHERE res_seatnum = 15;
```

Για να καταλάβουμε την διαφορά ξανά τυπώνουμε τους πίνακες.

Πίνακας travel_to

	to_tr_id	to_dst_id	to_arrival	to_departure
▶	1200	99	2019-12-28 17:25:30	2020-12-28 17:25:30
	1200	101	2019-12-27 17:30:30	2020-01-03 18:00:00
	1200	102	2019-12-27 18:30:30	2020-01-03 19:00:00
	1200	106	2019-12-27 16:30:30	2020-01-03 18:00:00
	1200	107	2019-12-27 15:30:30	2020-01-03 18:00:00
	1201	105	2023-01-04 09:00:00	2023-02-12 21:15:00
	1300	104	2018-07-30 06:00:30	2018-08-07 15:00:00
	1300	111	2018-08-07 21:30:30	2018-08-14 05:00:00
	1301	106	2023-03-01 23:30:00	2023-03-04 15:30:00
	1301	107	2023-02-13 22:00:00	2023-02-27 08:30:00

Διαγράφηκε το to_dst_id με 100 και έχει γίνει πρόσθεση με to_dst_id 99.

Πίνακας reservation

	res_tr_id	res_seatnum	res_name	res_lname	res_isadult
►	1200	1	Lucinda	Roach	ADULT
	1200	9	Edwin	Terrell	MINOR
	1200	10	Marvin	Morgan	ADULT
	1200	15	Giannis	Proctor	ADULT
	1200	16	Luther	Proctor	MINOR
	1201	2	Haris	Lamb	ADULT
	1201	17	Elsbeth	Schneider	MINOR
	1201	39	Georgiana	Franco	ADULT
	1201	50	Denzel	Smith	ADULT
	1300	9	Tara	Case	ADULT
	1300	21	Aleksander	Morris	MINOR
	1301	58	Mariam	King	ADULT
	1301	90	Robert	Gross	ADULT

Παρατηρούμε ότι διαγράφηκε η στήλη Zach και η Vanessa έχει ενημερωθεί σε Giannis.

Πίνακας logg

	log_AT	action1	table_act	it_lname	act_date
►	1	DELETE	reservation	Andrew	2023-09-16 00:56:23
	2	DELETE	travel_to	Phillips	2023-09-16 00:56:23
	3	UPDATE	reservation	Andrew	2023-09-16 00:56:23
	4	INSERT	travel_to	Andrew	2023-09-16 00:56:23

Που στο τέλος βλέπουμε στο πίνακα logg ποιος IT έχει κάνει την τροποποίηση.

Όπως αναφέραμε και παραπάνω έχουμε αλλάξει στο DELETE trigger τον IT του πίνακα travel_to για να έχουμε μία διαφορά. Παρόλο που ο IT στους πίνακες που αναφέρουμε στο word είναι ένας ο IT που αναφέρουμε. Η αλλαγή έγινε μόνο στο παράδειγμα.

3.1.4 Δημιουργία Trigger

3.1.4.2. Trigger ο οποίος θα αποτρέπει την αλλαγή της ημερομηνίας αναχώρησης, ημερομηνίας επιστροφής και του κόστους του ταξιδιού, αν έχουν ήδη γίνει κρατήσεις γι' αυτό.

Κώδικας:

```
DROP TRIGGER IF EXISTS prevent_trip_update;
DELIMITER $
CREATE TRIGGER prevent_trip_update
BEFORE UPDATE ON trip
FOR EACH ROW
BEGIN
    DECLARE reservation_count INT;

    -- Έλεγχος αν υπάρχει κρατήσεις για κάποιο ταξίδι
    SELECT COUNT(*) INTO reservation_count
    FROM reservation
    WHERE res_tr_id = NEW.tr_id;

    -- Αν υπάρχει κράτηση απορρίπτουμε την αλλαγή
    IF reservation_count > 0 THEN
        IF NEW.tr_departure != OLD.tr_departure OR
           NEW.tr_return != OLD.tr_return OR
           NEW.tr_cost != OLD.tr_cost THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Δεν επιτρέπεται η αλλαγή ημερομηνίας ή
            κόστος σε ταξίδια με κρατήσεις.';
        END IF;
    END IF;
END;
```

Πίνακας Reservation:

	res_tr_id	res_seatnum	res_name	res_lname	res_isadult
►	1200	1	Lucinda	Roach	ADULT
	1200	9	Edwin	Terrell	MINOR
	1200	10	Marvin	Morgan	ADULT
	1200	15	Vanessa	Proctor	ADULT
	1200	16	Luther	Proctor	MINOR
	1201	2	Haris	Lamb	ADULT
	1201	17	Elsbeth	Schneider	MINOR
	1201	39	Georgiana	Franco	ADULT
	1201	50	Denzel	Smith	ADULT
	1300	9	Tara	Case	ADULT
	1300	21	Aleksander	Morris	MINOR
	1300	32	Zach	Duffy	ADULT
	1301	58	Mariam	King	ADULT
	1301	90	Robert	Gross	ADULT

Πίνακας Trip (πριν ενεργοποιηθεί το Trigger)

	tr_id	tr_departure	tr_return	tr_maxseats	tr_cost	tr_br_code	tr_gui_AT	tr_drv_AT
▶	1200	2019-12-22 21:30:30	2020-01-03 18:00:00	110	349.90	20000	8147862271	97C9685C31
	1201	2023-01-04 09:00:00	2023-02-12 21:15:00	50	320.99	20001	831482CC74	710CC7A849
	1202	2021-02-18 23:00:00	2021-03-02 08:00:00	98	80.75	20002	CA3108032C	B1855A1B76
	1203	2024-05-12 18:00:00	2024-06-02 23:00:00	124	34.00	20003	8B207900AB	251A2C4988
	1204	2022-08-25 07:00:00	2022-09-18 15:00:00	90	560.50	20004	6C5C004CC3	72A3AB5128
	1300	2018-07-30 06:00:30	2018-08-14 05:00:00	89	224.50	20000	8C8166410C	AB74CA2361
	1301	2023-02-13 22:00:00	2023-03-04 15:30:00	105	1240.50	20001	0BB4BC6AB2	68B9B9909B
	1302	2017-06-19 07:00:00	2017-07-02 06:00:00	75	1119.99	20002	43A216CBB4	369935A4C4
	1303	2016-12-22 09:00:00	2017-01-15 12:00:00	60	125.80	20003	15C3A8C1C0	0337628780
	1304	2021-10-23 08:30:00	2021-11-24 18:00:00	122	1200.00	20004	C59C685619	62691A3677

Για να ενεργοποιήσουμε το **Trigger** θα χρειαστεί να κάνουμε update σε δεδομένα, αλλαγή ημερομηνίας (αναχώρησης ή επιστροφής) ή κόστος του ταξιδιού. Βέβαια, θα μπορούσαμε να αλλάξουμε και τα 2 δεδομένα.

Ένα παράδειγμα για ενεργοποίηση του Trigger:

```
update trip set tr_cost='50.50'
where tr_id='1200'
```

Σε αυτό το παράδειγμα παρατηρούμε ότι το id (**tr_id**) που έχουμε διαλέξει έχει reservation με αποτέλεσμα που θα πρέπει να ενεργοποιηθεί το trigger και να υπάρξει σχετικό μήνυμα.

Error Code: 1644 Δεν επιτρέπεται η αλλαγή ημερομηνίας ή κόστος σε ταξίδια με κρατήσεις.

300	18:28:35	insert into reservation_offers values ('Amelia','Moyer',NULL,'2500','182'). (Th...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
301	18:28:35	DROP TRIGGER IF EXISTS decreaseSalary	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist	0.000 sec
302	18:28:35	CREATE procedure getBranchDriverNum(IN AT_wrk_driver char(10),wrk_dri...	0 row(s) affected, 4 warning(s): 1681 Specifying number of digits for floating p...	0.015 sec
303	18:28:35	/* Πρώτο Procedure */ DROP PROCEDURE IF EXISTS getBranchDriver...	0 row(s) affected	0.000 sec
304	18:28:35	CREATE procedure getBranchDriverNum(IN AT_wrk_driver char(10),IN wrk...	0 row(s) affected, 3 warning(s): 1681 Specifying number of digits for floating p...	0.000 sec
305	18:28:35	/* Δεύτερο Trigger */ DROP TRIGGER IF EXISTS prevent_trip_update	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist	0.000 sec
306	18:28:35	CREATE TRIGGER prevent_trip_update BEFORE UPDATE ON trip FOR E...	0 row(s) affected	0.000 sec
307	18:28:35	CREATE TRIGGER prevent_trip_update BEFORE UPDATE ON trip FOR E...	Error Code: 1644 Δεν επιτρέπεται η αλλαγή ημερομηνίας ή κόστος σε τα...	

Μπορούμε να παρατηρήσουμε ότι το trigger δουλεύει κανονικά και πετάει το Error αν υπάρχει κράτηση.

	tr_id	tr_departure	tr_return	tr_maxseats	tr_cost	tr_br_code	tr_gui_AT	tr_drv_AT
▶	1200	2019-12-22 21:30:30	2020-01-03 18:00:00	110	349.90	20000	8147862271	97C9685C31

Συνεπώς δεν αλλάζει και η τιμή του συγκεκριμένου ταξιδιού λόγω του Trigger.

Ας εξετάσουμε την δεύτερη περίπτωση, δηλαδή να προσπαθήσουμε να κάνουμε κάποια ενημέρωση σε ταξίδι που **δεν** υπάρχει κάποια κράτηση:

```
update trip set tr_cost='50.50'
where tr_id='1202'
```

Για αυτή την λειτουργία θα χρησιμοποιήσουμε το **trip** με κώδικα (**tr_id**) που μπορούμε να δούμε και από το πίνακα **reservation** δεν υπάρχει κάποια κράτηση για το συγκεκριμένο ταξίδι.

Αλλάζουμε την τιμή του ταξιδιού από 80.75 σε 50.50:

Πίνακας Trip (σε περίπτωση που δεν υπάρχει κράτηση)

	tr_id	tr_departure	tr_return	tr_maxseats	tr_cost	tr_br_code	tr_gui_AT	tr_drv_AT
▶	1200	2019-12-22 21:30:30	2020-01-03 18:00:00	110	349.90	20000	8147862271	97C9685C31
	1201	2023-01-04 09:00:00	2023-02-12 21:15:00	50	320.99	20001	831482CC74	710CC7A849
	1202	2021-02-18 23:00:00	2021-03-02 08:00:00	98	50.50	20002	CA3108032C	B1855A1B76
	1203	2024-05-12 18:00:00	2024-06-02 23:00:00	124	34.00	20003	8B207900AB	251A2C4988
	1204	2022-08-25 07:00:00	2022-09-18 15:00:00	90	560.50	20004	6C5C004CC3	72A3AB5128
	1300	2018-07-30 06:00:30	2018-08-14 05:00:00	89	224.50	20000	8C8166410C	AB74CA2361
	1301	2023-02-13 22:00:00	2023-03-04 15:30:00	105	1240.50	20001	0BB4BC6AB2	68B9B9909B
	1302	2017-06-19 07:00:00	2017-07-02 06:00:00	75	1119.99	20002	43A216CBB4	369935A4C4
	1303	2016-12-22 09:00:00	2017-01-15 12:00:00	60	125.80	20003	15C3A8C1C0	0337628780
	1304	2021-10-23 08:30:00	2021-11-24 18:00:00	122	1200.00	20004	C59C685619	62691A3677

Πράγματι, αλλάζει το κόστος (tr_cost) του ταξιδιού με κώδικα (tr_id) 1202.

3.1.4.3 Trigger ο οποίος δεν θα επιτρέπει τη μείωση του μισθού ενός υπαλλήλου.

Κώδικας:

```
DROP TRIGGER IF EXISTS prevent_salary_reduction;
DELIMITER $
CREATE TRIGGER prevent_salary_reduction
BEFORE UPDATE ON Worker
FOR EACH ROW
BEGIN
    IF NEW.wrk_salary < OLD.wrk_salary THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Δεν επιτρέπεται η μείωση μισθού.';
    END IF;
END;
```

Επεξήγηση Trigger

Σε αυτή την άσκηση έχουμε ένα Trigger που θα απορρίπτει την περίπτωση που κάποιος μειώσει κάποιο μισθό κάποιου εργαζόμενου. Για να δουλέψει το Trigger, πρώτα θα υλοποιήσουμε μία περίπτωση που θα ενεργοποιήσει το Trigger για να δούμε αν πράγματι δουλεύει το Trigger.

Στην συνέχεια θα αυξήσουμε κάποιο μισθό ενός υπάλληλου για να καταλάβουμε ότι δεν εμποδίζει αυτήν την περίπτωση το συγκεκριμένο Trigger

Παρακάτω έχουμε το Πίνακα Worker με τους μισθούς.

Πίνακας Worker

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code
▶	0337628780	Nataniel	Shepherd	35000.50	20003
	03EC1106E8	Layton	Gibbons	25000.00	20000
	0BB4BC6AB2	Claire	Coleman	60000.50	20001
	15C3A8C1C0	Andreas	Potts	47500.50	20003
	197B244051	Arran	Bender	95500.20	20003
	2161C52254	Rose	Cruz	88000.40	20004
	251A2C4988	Haaris	Lucas	25000.00	20003
	26873C4924	Stephanie	Park	30500.50	20001
	369935A4C4	Corey	Orr	34000.50	20002
	4399691360	Scarlet	Chaney	45000.50	20002
	43A216CBB4	Heidi	Scott	55000.00	20002
	4BD720A79B	Gilbert	Campbell	27000.50	20003
	4C045A649A	Olivia	Camacho	38000.60	20003
	5258C7B8C6	Elodie	Carlson	87000.50	20002
	621B027416	Caoimhe	House	25000.00	20000
	62691A3677	Rupert	Sloan	42460.60	20004
	643CB89970	Josh	Molina	30000.50	20001
	68B9B9909B	Jackson	Davila	43000.60	20001
	6C5C004CC3	Marco	Hart	60000.40	20004
	6C738794A1	Lucia	Bray	27000.50	20001
	710CC7A849	Thomas	Butler	95500.00	20001
	72A3AB5128	Earl	Kemp	90500.50	20004
	737787B8B3	Samson	Hampton	78500.50	20000
	745066107B	Lilly-Rose	Wilkerson	98000.50	20002
	757006C2BB	Ellie-Mae	Barr	30500.20	20003
	8147862271	Dedan	Hogan	20000.50	20000
	820C85C87C	Kate	Keith	23500.00	20001
	831482CC74	Jimmy	Humphrey	76000.50	20001
	88244C2507	Gerard	Mccoy	55500.50	20004
	8B207900AB	Macie	Cobb	34000.00	20003
	8C8166410C	Dennis	Branch	40000.40	20000
	93C4B8CB6C	Elise	Meyer	40000.40	20004
	97C9685C31	Kaylum	Gallagher	35500.50	20000
	98872BAA72	Sandra	Koch	70500.50	20000
	AB74CA2361	Jerry	Wallace	42000.50	20000
	B1855A1B76	John	Stewart	50125.00	20002
	C59C685619	Gunilla	Bowen	45000.50	20004
	CA3108032C	Anoushka	Bridges	63125.00	20002
	D0D31BBC92	Alexander	Knapp	32000.00	20002

Για ενεργοποίηση του Trigger ας πάρουμε το παράδειγμα με **αριθμό ταυτότητας (wrk_AT)** τον Jerry Wallace. Βλέπουμε ότι έχει **μισθό (wrk_salary)** 42000.50.

Πρώτη Περίπτωση:

Μειώνουμε το μισθό του συγκεκριμένου υπάλληλου:

```
update worker
set wrk_salary='12000.50'
where wrk_AT='AB74CA2361'
```

221	18.26.29	CREATE procedure getBranchDriverNum(IN AT_wrk_driver char(10),IN wrk...	0 row(s) affected, 3 warning(s): 1681 Specifying number of digits for floating p...	0.000 sec
222	18.26.29	/* Δεύτερο Trigger */ DROP TRIGGER IF EXISTS prevent_trip_update	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist	0.000 sec
223	18.26.29	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON Work...	0 row(s) affected	0.000 sec
224	18.26.29	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON Work...	Error Code: 1644 Δεν επιτρέπεται η μείωση μισθού.	

Error Code: 1644 Δεν επιτρέπεται η μείωση μισθού.

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code
▶	AB74CA2361	Jerry	Wallace	42000.50	20000

Παρατηρούμε ότι δεν υπάρχει κάποια αλλαγή στο μισθό του συγκεκριμένου εργαζόμενου.

Δεύτερη περίπτωση:

Σε αυτή την περίπτωση αυξάνουμε το μισθό του υπάλληλου για να παρατηρήσουμε ότι ο Trigger ενεργοποιείται μόνο σε περιπτώσεις που μειώνουμε το μισθό.

Ο συγκεκριμένος εργαζόμενος (worker) έχει μισθό 42000.50 και θα το αλλάξουμε σε 62000.50.

```
update worker
set wrk_salary='62000.50'
where wrk_AT='AB74CA2361'
```

	wrk_AT	wrk_name	wrk_lname	wrk_salary	wrk_br_code
▶	AB74CA2361	Jerry	Wallace	62000.50	20000

Μπορούμε να επιβεβαιωθούμε ότι σε αυτήν την περίπτωση γίνεται η αύξηση κανονικά και δεν ενεργοποιείται ο Trigger.